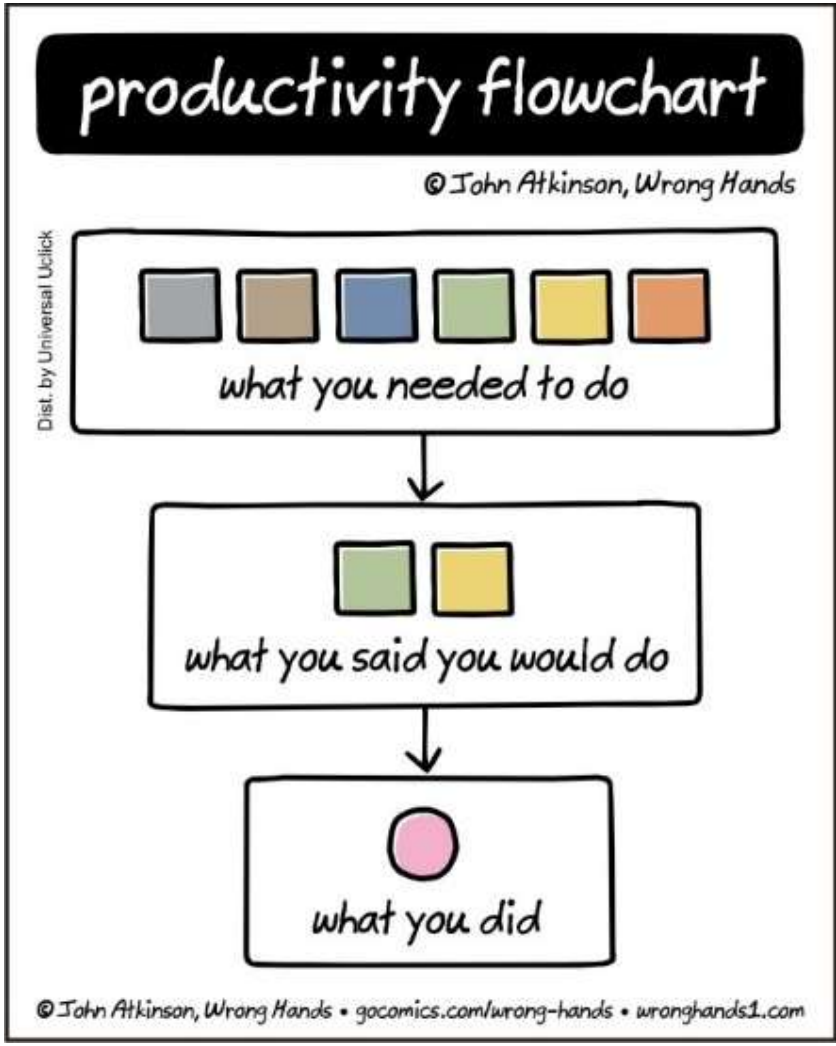
Tehnologii Web

procesarea datelor XML/HTML (I)

DOM

Document Object Model



Dr. Sabin Corneliu Buraga – profs.info.uaic.ro/~busaco/

„Regula de aur este că nu există reguli de aur.”

George Bernard Shaw

Cum putem prelucra documentele XML?

preambul

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE html>
<!-- pentru redarea datelor, se recurge la o foaie de stiluri CSS -->
<?xml-stylesheet type="text/css" href="game.css" ?>
```

nod de tip comentariu

instrucțiune de procesare

```
    title>
    <platform type="tablet">Android</platform>
    <platform min-version="9" type="tablet">iOS</platform>
    <platform min-version="10"
    <url>...</url>
    <player>
        <identity
            <first-name>Sabin</first-name>
            <last-name>Buraga</last-name>
            <!-- ... -->
        </identity>
        <points>30374</points>
    </player>
</game>
```

nod de tip atribut

tag de început

tag de final

element XML marcat cu *tag* de început și *tag* de final

nod de tip text (*characters*)

Tipuri de procesări XML

procesare manuală

e.g., expresii regulate 😢

Tipuri de procesări XML

procesare obiectuală

DOM (*Document Object Model*)
non-DOM

Tipuri de procesări XML

procesare condusă de evenimente

SAX (*Simple API for XML*)
XPP (*XML Pull Parsing*)

vezi cursul viitor

Tipuri de procesări XML

procesare simplificată

Simple XML

vezi cursul viitor

Tipuri de procesări XML

procesare particulară

via API-uri specializate pentru a prelucra
tipuri de documente specifice – *e.g.*, RSS, SOAP, SVG,…

Procesoare (analizoare) XML
fără validare

verifică doar dacă documentul
este bine-formatat (*well formed*)

Expat, libxml, MSXML,...

Procesoare (analizoare) XML
cu validare

verifică dacă documentul este valid,
conform unei metode de validare – *e.g.*, DTD

Apache Xerces, JAXP, libxml, MSXML,...

Modelul DOM

introducere
niveluri de specificare
interfețe DOM
DOM Core
implementări
exemple
DOM direct în navigatorul Web

dom: intro

Scop:
procesarea obiectuală – standardizată –
a documentelor XML și/sau HTML

dom: caracterizare

Interfață de programare a aplicațiilor (API) abstractă pentru XML/HTML

dom: caracterizare

Interfață de programare a aplicațiilor (API) abstractă pentru XML/HTML

independentă de platformă și limbaj de programare

standardizată de Consorțiul Web

www.w3.org/DOM/DOMTR

dom: caracterizare

Definește o structură logică arborescentă
a documentelor XML

document ≡ ierarhie a unui set de obiecte
pe baza cărora se pot accesa/modifica noduri și date XML

dom: niveluri de specificare

DOM 1 (1998)

www.w3.org/TR/REC-DOM-Level-1/

DOM Core pentru XML – funcționalități de bază

DOM HTML pentru procesarea standardizată
a paginilor Web – uzual, la nivel de client (*browser*)

dom: niveluri de specificare

DOM 2 (2001)

www.w3.org/TR/REC-DOM-Level-2/

recomandări multiple privind diverse aspecte: spații de nume XML, aplicare de stiluri CSS, răspuns la evenimente etc.

dom: niveluri de specificare

DOM 3 (2004)

www.w3.org/TR/DOM-Level-3-Core/

funcționalități specifice oferite de module (unele deja standardizate)

XPath, traversare a arborelui, validare, încărcare și salvare (asincrone),...

dom: niveluri de specificare

DOM 4 (2015)

www.w3.org/TR/dom/

restructurarea unor interfețe + noi funcționalități

dom: niveluri de specificare

DOM *Living Standard*

dom.spec.whatwg.org

specific HTML

în continuă dezvoltare
(cea mai recentă actualizare: 23 martie 2021)

Markup to test (permalink, save, upload, download, hide):

```
<!DOCTYPE html>
<html><head><title>DOM</title></head><body><article><h1 class="h">Web</h1>
<img src="image" alt="Web"/>Document <em>Object</em> Model</article></body>
```

DOM view (hide, refresh):

```
DOCTYPE: html
HTML
  HEAD
    TITLE
      #text: DOM
  BODY
    ARTICLE
      H1 class="h"
        #text: Web
      #text:
      IMG src="image" alt="Web"
      #text: Document
      EM
        #text: Object
      #text: Model
```

în cazul HTML, numele elementelor sunt disponibile cu litere mari (*capitals*)

un document HTML și arborele DOM corespunzător reprezentat via Live DOM Viewer

software.hixie.ch/utilities/js/live-dom-viewer/

Markup to test (permalink, save, upload, download, hide):

```
<!DOCTYPE html>
<html><head><title>DOM</title></head><body><article><h1 class="h">Web</h1>
<img src="image" alt="Web"/>Document <em>Object</em> Model</article></body>
```

DOM view (hide, refresh):

```
DOCTYPE: html
HTML
  HEAD
    TITLE
      #text: DOM
  BODY
    ARTICLE
      H1 class="h"
        #text: Web
      #text:
      IMG src="image" alt="Web"
      #text: Document
      EM
        #text: Object
      #text: Model
```

De ce apare
și acest nod?

un document HTML și arborele DOM corespunzător reprezentat via Live DOM Viewer

software.hixie.ch/utilities/js/live-dom-viewer/

dom: interfețe

Modalitate abstractă de accesare și de modificare a reprezentării interne a unui document XML

dom: interfețe

Modalitate abstractă de accesare și de modificare a reprezentării interne a unui document XML

datele sunt încapsulate în obiecte, ascunse și/sau protejate de prelucrarea externă directă

dom: interfețe

Nu implică o implementare concretă, particulară

DOM oferă interfețe
independente de implementare pentru
accesarea/procesarea datelor

dom: interfețe

Interfețele sunt definite via
IDL (*Interface Description Language*)

specificația curentă (2018): **www.omg.org/spec/IDL/**

dom: interfețe – IDL

Definește tipurile de obiecte
prin specificarea interfețelor acestora
(proprietăți + metode publice)

pur declarativ

dom: interfețe – IDL

Oferă suport pentru moștenire multiplă prin intermediul interfețelor

declară module, interfețe, metode, tipuri, atribute, excepții, constante

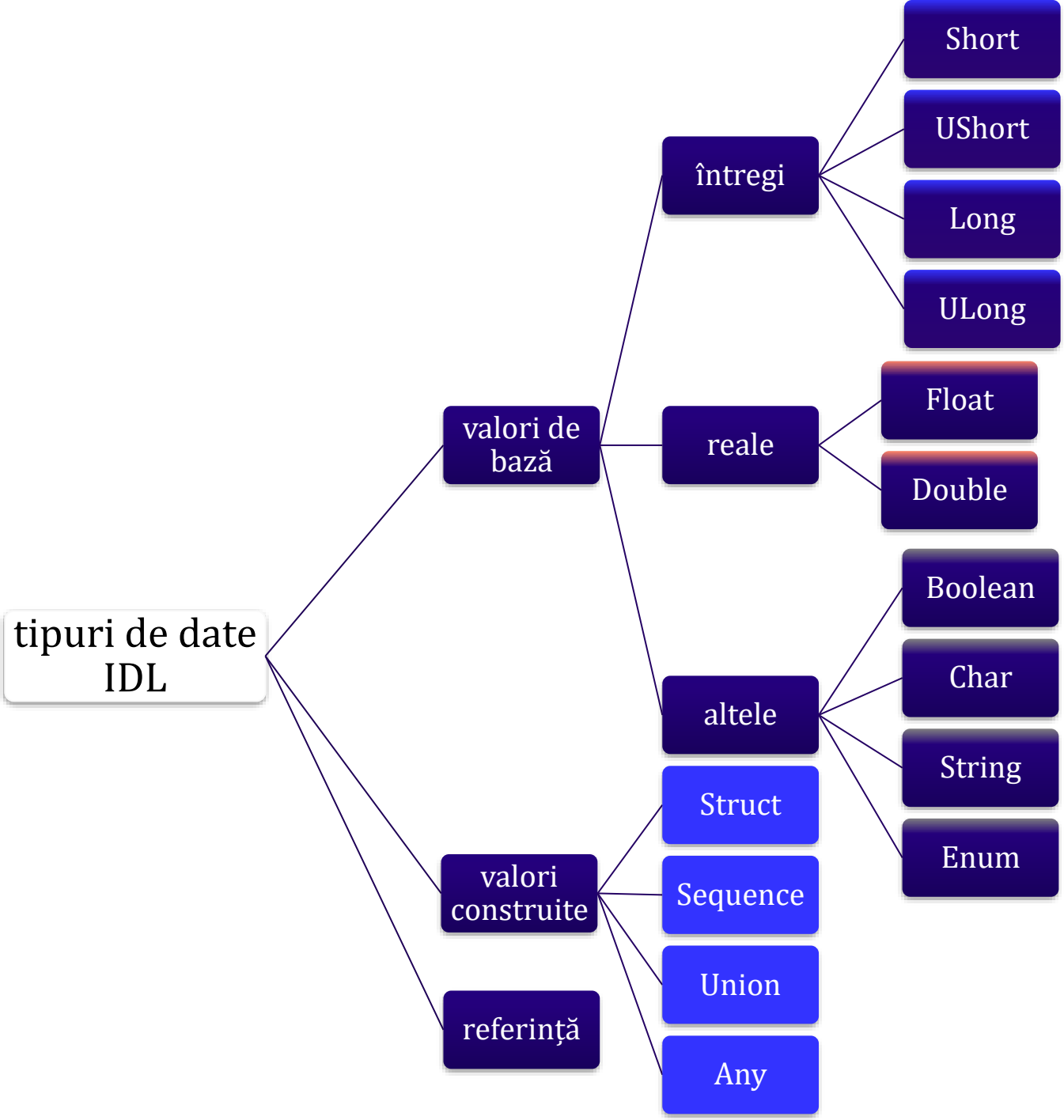
dom: interfețe – IDL

Oferă suport pentru moștenire multiplă prin intermediul interfețelor

specificații oficiale:

www.w3.org/TR/WebIDL-1/ (*W3C Recommendation*, 2016)

heycam.github.io/webidl/ (în lucru, 11 martie 2021)


tipuri de date IDL
valori de bază
valori construite
referință
întregi
reale
altele
Short
UShort
Long
ULong
Float
Double
Boolean
Char
String
Enum
Struct
Sequence
Union
Any

dom: interfețe – exemplu

Declararea interfeței vizând lista de noduri (elemente, atribute, caractere, comentarii,…) ale arborelui XML: **NodeList**

```
interface NodeList {
  getter Node? item (unsigned long index);
  readonly attribute unsigned long length;
};
```

dom: interfețe – exemplu

```
interface NodeList {
  getter Node? item (unsigned long index);
  readonly attribute unsigned long length;
};
```

metodă de preluare a informațiilor privind un nod dintr-o listă de noduri

la intrare: un parametru de tip întreg lung fără semn

rezultat: o valoare de tip **Node**

proprietate *read-only* de tip întreg lung fără semn

dom: interfețe – exemplu

Specificarea interfeței privind un atribut XML:
Attr

```
interface Attr : Node {
  readonly attribute DOMString name;
  readonly attribute boolean specified;
  attribute DOMString value;
};
```

dom: interfețe – exemplu

Attr extinde Node

```
interface Attr : Node {
  readonly attribute DOMString name;
  readonly attribute boolean specified;
  attribute DOMString value;
};
```

3 proprietăți

name desemnează numele atributului XML
specified indică dacă atributul a fost specificat efectiv
value reprezintă valoarea curentă a atributului

dom: core

Un document ≡ ierarhie de **obiecte de tip nod** care pot implementa interfețe (specializate)

dom: core

Un document ≡ ierarhie de **obiecte de tip nod** care pot implementa interfețe (specializate)

nodurile posedă descendenți ori sunt noduri frunză

dom: core

Un document ≡ ierarhie de **obiecte de tip nod**
care pot implementa interfețe (specializate)

nodurile posedă descendenți ori sunt noduri frunză

parcurgerea arborelui se efectuează
în preordine, în adâncime (*depth-first*) – ***tree order***

dom: core

Accesul la date
– liste de noduri, atribute, valori,… –
se realizează recurgându-se la metodele specifice fiecărui tip de noduri ale arborelui

dom: core – tipuri de noduri

Document	**Element**, **ProcessingInstruction**, **Comment**, **DocumentType**
Document Fragment	**Element**, **ProcessingInstruction**, **Comment**, **Text**, **CDATASection**,…
Element	**Element**, **Text**, **Comment**, **CDATASection**, **EntityReference**,…
Attr	**Text**, **EntityReference**
Text	– (nod frunză al arborelui DOM)

dom: core

Interfețe fundamentale:

DOMException

gestionează setul de excepții de procesare

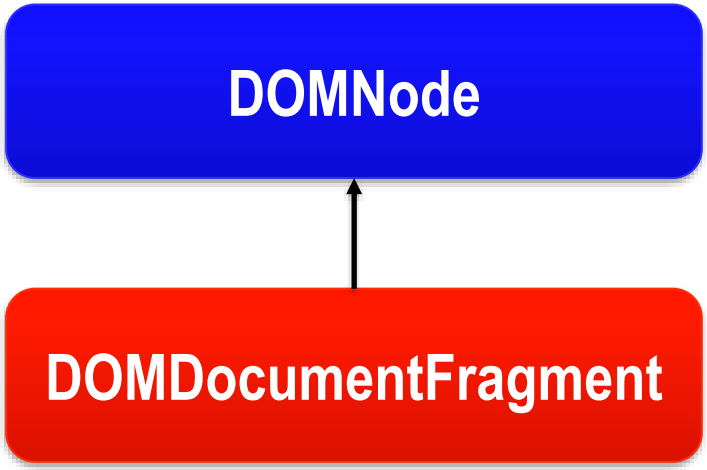
dom: core

Interfețe fundamentale:

DocumentFragment : Node

acces la fragmente de arbore

(obiect minimal fără nod rădăcină)



dom: core

Interfețe fundamentale:

Document

furnizează accesul la document
pentru consultare și/sau modificare

dom: core

Interfețe fundamentale:

Document

proprietăți folositoare:
doctype – indică tipul de document (DTD)
documentElement – desemnează elementul-rădăcină

Metode esențiale ale interfeței Document

createElement() – creează un nod de tip element

createTextNode() – creează nod de tip text (conținut efectiv)

createAttribute() – creează un nod de tip atribut (**Attr**)

getElementsByTagName() – oferă lista elementelor având un anumit nume (*tag*)

getElementById() – furnizează elementul identificat prin valoarea atributului **id**

importNode() – importă un nod din alt document prin clonarea datele acestui nod

etc.

dom: core

Interfețe fundamentale:
Node
permite accesul la nodurile arborelui

tipuri de noduri
– *e.g.*, Document, Element,
CharacterData (Text, Comment, CDATASection),
DocumentFragment, DocumentType, EntityReference,… –
prelucrate în mod unitar

Proprietăți importante ale interfeței Node

nodeName – furnizează numele nodului curent

nodeValue – oferă valoarea curentă a nodului

nodeType – furnizează tipul nodului: element, atribut,…

parentNode – desemnează nodul părinte al nodului curent

parentElement – specifică elementul părinte al nodului

Tipuri de noduri (proprietatea **nodeType**)	Valoare
ELEMENT_NODE	1
ATTRIBUTE_NODE	2
TEXT_NODE	3
CDATA_SECTION_NODE	4
ENTITY_REFERENCE_NODE	5
ENTITY_NODE	6
PROCESSING_INSTRUCTION_NODE	7
COMMENT_NODE	8
DOCUMENT_NODE	9
DOCUMENT_TYPE_NODE	10
DOCUMENT_FRAGMENT_NODE	11
NOTATION_NODE	12

Proprietăți importante ale interfeței Node

childNodes – oferă lista nodurilor copil ale nodului curent

firstChild – desemnează primul copil ale nodului curent

lastChild – specifică ultimul copil ale nodului curent

previousSibling – reprezintă precedentul nod (*sibling*) aflat pe același nivel cu cel al nodului curent

nextSibling – indică următorul nod (*sibling*) aflat pe același nivel cu cel al nodului curent

etc.

Metode utile oferite de interfața Node

insertBefore() – inserează un nod înaintea celui curent

cloneNode() – clonează datele nodului curent

compareDocumentPosition() – compară poziția nodului curent cu alt nod în cadrul documentului

isEqualNode() – testează dacă nodul curent este echivalent cu alt nod (același tip de nod, aparține aceluiași spațiu de nume, are aceeași valoare, ambele noduri au același număr de copii, fiecare nod copil are același index pentru ambele noduri etc.) – vezi și isSameNode()

contains() – verifică dacă nodul include un nod anume

Metode utile oferite de interfața Node

hasChildNodes() – verifică dacă nodul curent are noduri copil

appendChild() – adaugă un nod copil

replaceChild() – substituie un nod copil

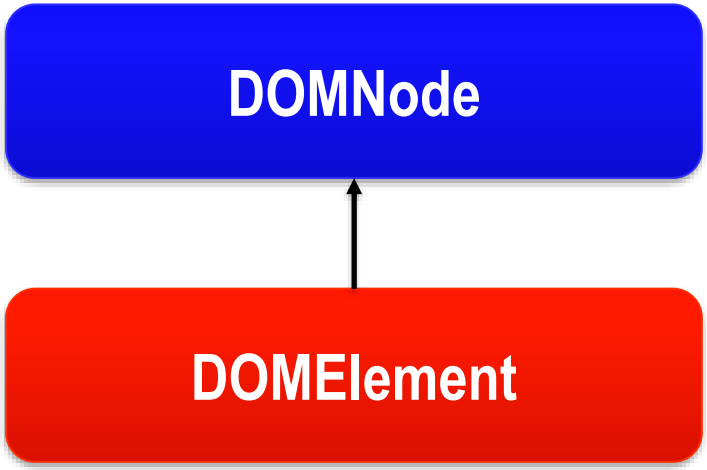
removeChild() – elimină un nod copil

dom: core

Interfețe fundamentale:

Element

facilitează accesul la elementele unui document XML



Proprietăți folositoare ale interfeței Element

tagName – furnizează numele elementului (*tag* de început)

id – specifică valoarea identificatorului (adică valoarea atributului **id**)

childNodes – oferă lista nodurilor copil ale elementului

firstChild lastChild – indică primul/ultimul nod copil

attributes – oferă lista atributelor asociate elementului

nextElementSibling – indică următorul element (*sibling*) aflat pe același nivel din arbore cu elementul curent

previousElementSibling – idem, dar pentru cel precedent

Metode utile specificate de interfața Element

getAttribute() – furnizează valoarea unui atribut asociat elementului curent

getAttributeNode() – oferă nodul de tip atribut

setAttribute() – stabilește valoarea unui atribut

removeAttribute() – elimină un atribut asociat elementului

hasAttribute() – verifică existența unui atribut specific

Metode utile specificate de interfața Element

getAttribute() – furnizează valoarea unui atribut asociat elementului curent

getAttributeNode() – oferă nodul de tip atribut

setAttribute() – stabilește valoarea unui atribut

removeAttribute() – elimină un atribut asociat elementului

hasAttribute() – verifică existența unui atribut specific

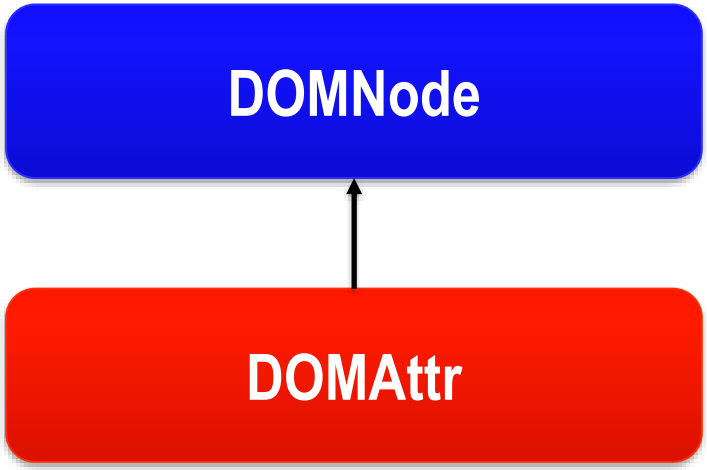
pentru a prelucra atribute aparținând
unui spațiu de nume XML dat,
se vor folosi metode sufixate de NS
e.g., getAttributeNS() setAttributeNS() etc.

dom: core

Interfețe fundamentale:

Attr : Node

modelează atributele unui element



dom: core

Interfețe fundamentale:

NodeList – acces la colecții de noduri via indecși (vector)

NamedNodeMap – acces pe baza cheilor (tablou asociativ)
în cazul HTML, pentru liste de atribute

dom: core

Interfețe fundamentale:

NodeList + NamedNodeMap

proprietate: length – lungimea listei de noduri

metode: item() getNamedItem() getNamedItemNS() setNamedItem() removeNamedItem() și altele

dom: core

Alte interfețe de interes:

ParentNode – nodul părinte specific unui nod
ChildNode – nodul copil aparținând unui nod
Elements – colecție de noduri de tip Element

dom: acces la noduri – xpath

Accesarea datelor dintr-un document

XPath 1.0 – recomandare a Consorțiului Web (1999)

versiuni ulterioare:
XPath 2 (2010), XPath 3 (2014, 2017)

www.w3.org/TR/xpath/all/

dom: acces la noduri – xpath

Expresia XPath – construcție de bază

evaluarea se realizează în funcție de context:

un nod al documentului XML

poziție

o funcție (predefinită sau specificată de programator)

o declarație a unui spațiu de nume

dom: acces la noduri – xpath

Expresia XPath – construcție de bază

în urma evaluării expresiei, e oferit un obiect:

o mulțime de noduri (*node-set*)

o valoare logică – *true/false*

un număr (*float*)

un șir de caractere

dom: xpath – operatori

Descendent /

/html/body/article
/game/platform

dom: xpath – operatori

Traversare recursivă //

//platform
//div/a

nodurile-copil sunt traversate în ordinea dată de algoritmul DFS (*depth-first search*)

dom: xpath – operatori

Wildcard *

selectează toate elementele copil ale nodului aflat în contextul curent (*context node*)

/game/*/identity

player/*

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml" lang="ro" xml:lang="ro">
  ▶ <head>…</head>
  ▼ <body itemscope itemtype="http://schema.org/WebPage">
    ▶ <header>…</header>
    ▼ <article>
      ▶ <p>…</p>
      ▶ <aside class="menu" lang="en" xml:lang="en">…</aside>
      ▶ <nav>…</nav>
...   ▶ <div id="past">…</div> == $0
      ▶ <div id="present">…</div>
      ▶ <div id="future">…</div>
      ▶ <section>…</section>
      </article>
    ▶ <footer>…</footer>
    </body>
  </html>
```

html body article div#past

Console

top ▼ Filter

```
> $x ("/html/body/*/div")
<· ▼ (3) [div#past, div#present, div#future]
```

/html/body/*/div

evaluarea expresiilor XPath în consola *browser*-ului Web via obiectul **$x**

dom: xpath – operatori

Nodul curent **.**

./player

dom: xpath – operatori

Atribut @

platform/@min-version

table/@*

dom: xpath – operatori

Spațiu de nume :

svg:*

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml" xmlns:svg="http://www.w3.org/2000/svg">
 <head><title>HTML + SVG</title></head>
 <body>
  <h1>HTML + SVG</h1>

  <!-- Elemente și atribute SVG incluse în documentul HTML,
       recunoscute pe baza spațiului de nume SVG -->
  <svg:svg width="500" height="200">
   <svg:rect …/>
   <svg:text …>SVG</svg:text>
   <svg:circle cx="400" cy="150" r="33" style="fill: yellow;" />
  </svg:svg>

  <p>De utilizat un navigator Web oferind suport nativ pentru SVG.</p>
 </body>
</html>
```

evaluarea expresiei **//svg:*** conduce la obținerea a 4 noduri:

```
<svg:svg xmlns:svg="http://www.w3.org/2000/svg" …><svg:rect …/>…</svg:svg>
<svg:rect xmlns:svg="http://www.w3.org/2000/svg" …/>
<svg:text xmlns:svg="http://www.w3.org/2000/svg" …>SVG</svg:text>
<svg:circle xmlns:svg="http://www.w3.org/2000/svg" …/>
```

dom: xpath – operatori

Filtru/index [ ]

/game/platform[2]
//li[@class = "lab"]

forma generală: [*expresie*]

```
▼<section id="week1">
  ▶<h2>…</h2>
  ▼<ul>
    ▶<li class="lecture" itemscope itemtype="http://schema.org/CreativeWork" itemid="#web-lecture0">…</li>
    ▶<li class="lecture" itemscope itemtype="http://schema.org/CreativeWork" itemid="#web-lecture1">…</li>
    ▶<li class="bib">…</li>
…   ▼<li class="lab"> == $0
      ▼<p>
        "Laborator: "
        <a href="https://www.w3.org/html/" title="Detalii despre HTML (HyperText Markup Language)">Limbajul de
        marcare HTML</a>
        " - de studiat "
      ▶<span itemscope itemtype="http://schema.org/CreativeWork">…</span>
        " și "
        <a href="https://www.html-5-tutorial.com/" title="Un tutorial privitor la HTML">HTML5 - A Tutorial for
        Beginners</a>
      </p>
    ▶<ul>…</ul>
```

html body article div#past section#week1 ul li.lab

//li[@class = "lab"]

Console

top ▼ | Filter | Default levels ▼ | ☑ Group similar

```
> $x ('//li[@class="lab"]')
<  ▼(13) [li.lab, li.lab, li.lab, li.lab, li.lab, li.lab, li.lab, li.lab, li.lab, li.lab, li.lab, li.lab, li.lab]
```

13 noduri **<li>** (de tip Element) având ca valoare a atributului **class** șirul de caractere **"lab"**

dom: xpath – operatori

Pentru valori logice și numerice pot fi folosiți operatorii uzuali:

or and = != <= < >= >

+ - * div mod

comparațiile de șiruri sunt *case sensitive*

dom: xpath – axe

Nodul curent (*context node*)	self
Nodul părinte	parent
Nodurile copil	child
Nodurile descendente	descendant
Nodurile de tip atribut	attribute
Nodurile spațiilor de nume	namespace

dom: xpath – axe

Nodurile ascendente	ancestor
Nodurile ascendente ori nodul curent	ancestor-or-self
Nodurile descendente ori nodul curent	descendant-or-self

```
▼<section id="week1">
  ▶<h2>…</h2>
  ▼<ul>
    ▶<li class="lecture" itemscope itemtype="http://schema.org/CreativeWork" itemid="#web-lecture0">…</li>
    ▼<li class="lecture" itemscope itemtype="http://schema.org/CreativeWork" itemid="#web-lecture1">
      ▶<aside class="menu">…</aside> == $0
      ▶<p>…</p>
      ▶<div class="terms" itemprop="keywords" lang="en" xml:lang="en">…</div>
```

html body article div#past section#week1 ul li.lecture aside.menu

Styles Event Listeners DOM

Console

top

Ce rezultat oferă expresiile XPath?

//section/descendant::aside

//section/descendant::*

```
> $x ('//section/descendant::aside')
  (16) [aside.menu, aside.menu, aside.menu, aside.menu, aside.menu, aside.menu, aside.menu, aside.menu, aside.menu, aside.menu, aside.menu, aside.menu, aside.menu, aside.menu, aside.menu, aside.menu]
> $x ('//section/descendant::*')
  (676) [h2, ul, li.lecture, p, a, span, div.terms, li.lecture, aside.menu, a, p, a, span, div.terms, li.bib, p, span, a, span,
```

xPath tester | XPath | XQuery | XSLT | Validate

XPath:

/game/player/identity/preceding::*

XML: Test | Save | Color | Format | Fix

```
<?xml version="1.0" encoding="UTF-8"?>
<game>
  <title>Angry Profs</title>
  <platform type="tablet">Android</platform>
  <platform min-version="9" type="tablet">iOS</platform>
  <platform min-version="10">Windows</platform>
  <url>...</url>
  <player>
    <identity>
      <first-name>Sabin</first-name>
      <last-name>Buraga</last-name>
      <!-- ... -->
    </identity>
    <points>30374</points>
  </player>
</game>
```

rezultatul expresiei XPath
/game/player/identity/preceding::*

```
<title>Angry Profs</title>

<platform type="tablet">Android</platform>

<platform min-version="9" type="tablet">iOS</platform>

<platform min-version="10">Windows</platform>

<url>...</url>
```

folosind axa **preceding** accesăm nodurile precedente

un instrument Web de testare: **www.xpathtester.com/xpath**

dom: xpath – sintaxa prescurtată

self	≡	.
parent	≡	..
attribute	≡	@
namespace	≡	:

```
../first-name
//child::*[@min-version > 9]
```

dom: xpath – funcții

Noduri: id(), position(), count(), name(), namespace-uri(), last(), …

Tipuri de noduri: node(), text(), comment(), processing-instruction()

Șiruri: concat(), starts-with(), contains(), substring(), string-length(), translate(), …

Booleeni: not(), true(), false()

Numere: sum(), round(), floor(), number(), …

numărarea nodurilor ce satisfac o anumită condiție

count(/game/platform[@min-version >= 9])

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <game>
3    <title>Angry Profs</title>
4    <platform type="tablet">Android</platform>
5    <platform min-version="9" type="tablet">iOS</platform>
6    <platform min-version="10">Windows</platform>
7    <url>...</url>
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL XPath Results

```
XPath Query: count(/game/platform[@min-version >= 9])

[Result]: 2
```

extensia XML Tools pentru Visual Studio Code

dom: xpath – funcții

count(//a[@href]) = count(//a) ▸ *true*

toate elementele **<a>** includ atributul **href**

count(//li[@class]) = count(//li) ▸ *false*

există elemente **<li>** care nu au specificat atributul **class**

```
> $x ('count(//a[@href]) = count(//a)')
<· true
> $x ('count(//li[@class]) = count(//li)')
<· false
```

```
▶<section id="week4">…</section>
▼<section id="week5">
  ▶<h2>…</h2>
  ▼<ul>
    ▶<li class="lecture" itemscope itemtype="http://schema.org/
    CreativeWork" itemid="#web-lecture5">…</li>
    ▼<li class="demo">
...   ▼<p> == $0
        "Demo: "
        <a href="demos/php/php.zip" title="Arhivă .zip">Exemple
        programe PHP</a>
        " (arhivă "
        <code>zip</code>
        ", 41.5K)"
      </p>
    </li>
    ▶<li class="bib">…</li>
    ▶<li class="tools">…</li>
```

html body article div#past section#week5 ul li.demo p

Styles Event Listeners DOM Breakpoints Properties Accessibility

Console

top ▼ Filter Default levels ▼ Group similar

```
> $x ('//*[contains(text(), "Demo")]')
<· ▼(14) [p, p, p, p, p, p, p, p, p, p, p, p, p, p]
```

//*[contains(text(), "Demo")]

furnizarea tuturor nodurilor care conțin șirul **"Demo"**

```
<projects>
 <project class="M">
  <title>Minunat</title>
  <desc>Un proiect minunat!</desc>
  <stud>3</stud>
  <url>http://minunat.info/</url>
 </project>

 <project class="B">
  <title>Minunat--</title>
  <desc>Un proiect care nu e minunat...</desc>
  <stud>2</stud>
 </project>
</projects>
```

media numărului de studenți alocați per proiect:

sum(/projects/project/stud/text()) div count(/projects/project/stud)

```
XPath Query: sum(/projects/project/stud/text()) div count(/projects/project/stud)

[Result]: 2.5
```

evaluarea expresiilor XPath
direct în linia de comandă

```
<projects>
 <project class="M">
  <title>Minunat</title>
  <desc>Un proiect minunat!</desc>
  <stud>3</stud>
  <url>http://minunat.info/</url>
 </project>

 <project class="B">
  <title>Minunat--</title>
  <desc>Un proiect care nu e minunat...</desc>
  <stud>2</stud>
 </project>
</projects>
```

instrumentul xmllint (folosește biblioteca LibXML)
oferit de distribuțiile GNU/Linux + macOS

```
xmllint projects-dtd.xml
--xpath "substring(/projects/project[1]/desc/text(), 4, 7)"
```

(în loc de) pauză

dom: xpath – programare

Exemplificare:

folosirea XPath pentru procesarea unui flux de știri RSS (*news feed*) într-un program PHP
ce recurge la clasele DOMDocument, DOMXpath, DOMNode

de consultat
arhiva exemplelor

```
<rss xmlns:dc="http://purl.org/dc/elements/1.1/" version="2.0">
 <channel>
  <title>InfoQ</title>
  <link>https://www.infoq.com</link>
  <description>InfoQ feed</description>
  <item>
   <title>Java News Roundup - Week of March 22nd, 2021</title>
   <link>https://www.infoq.com/news/2021/03/java-news-roundup-mar22-2021/</link>
   <description>&lt;p&gt;News in the OpenJDK...&lt;/p&gt;</description>
   <category>Java</category>
   <category>Eclipse</category>
   <category>Open JDK</category>
   <category>Architecture &amp; Design</category>
   <category>Development</category>
   <category>news</category>
   <pubDate>Mon, 29 Mar 2021 05:45:00 GMT</pubDate>
   <guid>https://www.infoq.com/news/2021/03/java-news-roundup-mar22-2021/</guid>
   <dc:creator>Ben Evans</dc:creator>
   <dc:date>2021-03-29T05:45:00Z</dc:date>
   <dc:identifier>/news/2021/03/java-news-roundup-mar22-2021/en</dc:identifier>
  </item>
  <!-- alte noutăți... (i.e., elemente <item>) -->
 </channel>
</rss>
```

structura generală a unui flux RSS poate fi studiată la **www.rssboard.org**

```
define ('FEED', 'https://feed.infoq.com/');  // adresa fluxului de ştiri RSS
define ('XPATH', '/rss/channel/item');       // expresia XPath utilizată

try {
  $dom = new DomDocument ();
  $dom->load (FEED); // încărcăm documentul XML
  $xpath = new DOMXpath ($dom);
  // baleiăm însemnările (aici, elementele <item>)
  foreach ($xpath->query (XPATH) as $news) {
    // preluăm titlul fiecărei ştiri: valoarea elementului <title>
    echo '<p>Noutate: <em>' .
      $news->getElementsByTagname('title')->item(0)->nodeValue . '</p>';
    echo '<p>Calea nodului: <code>' .
      $news->getElementsByTagname('title')->item(0)->getNodePath() .
      '</code></p>';
  }
}
catch (RuntimeException $e) {
  echo $e->getMessage ();
}
```

metodă din DOMNode

Date RSS de la InfoQ

Noutate: *Java News Roundup - Week of March 22nd, 2021*

Calea nodului: `/rss/channel/item[1]/title`

Noutate: *.NET News Roundup - Week of March 22th, 2021*

Calea nodului: `/rss/channel/item[2]/title`

Noutate: *AWS Releases Health Aware Providing Automated Health Alerts for Accounts*

Calea nodului: `/rss/channel/item[3]/title`

Noutate: *Article: Platform Engineering As A (Community) Service*

Calea nodului: `/rss/channel/item[4]/title`

Noutate: *Rust 1.51 Stabilizes Const Generics MVP, Improves Cargo and Compile Times*

un posibil rezultat al procesării datelor RSS

Calea nodului: `/rss/channel/item[5]/title`

Metoda **query()** a clasei DOMXpath oferă lista nodurilor determinate în urma evaluării expresiei XPath (se întoarce o instanță de tip NodeList)

Pentru a obține rezultate de alt tip – de pildă, evaluarea unei expresii XPath poate fi de tip întreg –, se utilizează metoda **evaluate()**

```
$items = $xpath->query (XPATH);
echo '<p>Total apariții de elemente <code>&lt;item&gt;</code>:</p>';
echo '<p>Folosind proprietatea length (DOMNodeList): ' .
  $items->length . '</p>';
echo '<p>Folosind funcția XPath count(): ' .
  $xpath->evaluate ("count(" . XPATH . ")") . '</p>';
```

count(/rss/channel/item)

Total apariții de elemente <item>:

Folosind proprietatea length (DOMNodeList): 15

Folosind funcția XPath count(): 15

dom: implementări

DOMDocument – clasă PHP
php.net/manual/en/book.dom.php

HXT (*Haskell XML Toolbox*) – procesări DOM în Haskell
wiki.haskell.org/HXT

JAXP (*Java Architecture for XML Processing*)
parte integrantă din J2SE (javax.xml.*)
docs.oracle.com/javase/tutorial/jaxp/

jsdom – modul Node.js: **github.com/jsdom/jsdom**

JSXML – bibliotecă JavaScript: **jsxml.net**

dom: implementări

QDOM – la Qt (C++): **doc.qt.io/qt-5/qdomdocument.html**

libxml – procesor XML scris în C: **xmlsoft.org**
baza unor biblioteci pentru C++, Perl, PHP, Python, Ruby,…

org.w3c.dom.Document – interfață DOM (Android)
developer.android.com/reference/org/w3c/dom/Document.html

script::dom – modul Rust: **doc.servo.org/script/dom/**

TinyXML2 – bibliotecă de procesare DOM pentru C++
github.com/leethomason/tinyxml2

dom: implementări

Xerces DOM API – proiect axat pe procesări XML (C++ și Java): **xerces.apache.org**

XmlDocument – clasă .NET Framework (C# *et al.*)
docs.microsoft.com/en-us/dotnet/api/system.xml.xmldocument

XMLDocument – clasă Objective-C / Swift
developer.apple.com/documentation/foundation/xmldocument

XML::DOM – modul Perl: **search.cpan.org/perldoc?XML::DOM**
bazat pe procesorul Expat: **libexpat.github.io**

xml.dom – modul Python: **docs.python.org/3/library/xml.dom.html**

avansat

dom: api-uri particulare (exemple)

node-xmpp – biblioteca Node.js (JavaScript) pentru XMPP
github.com/xmppjs/xmpp.js

OpenDocument Format (ODF) SDKs – procesarea documentelor ODF în C#, Java, PHP, Python,…
opendocumentformat.org/developers/

Open Street Map API – acces la datele XML oferite de serviciul cartografic liber OpenStreetMap
wiki.openstreetmap.org/wiki/OSM_XML
wiki.openstreetmap.org/wiki/API

avansat

dom: api-uri particulare (exemple)

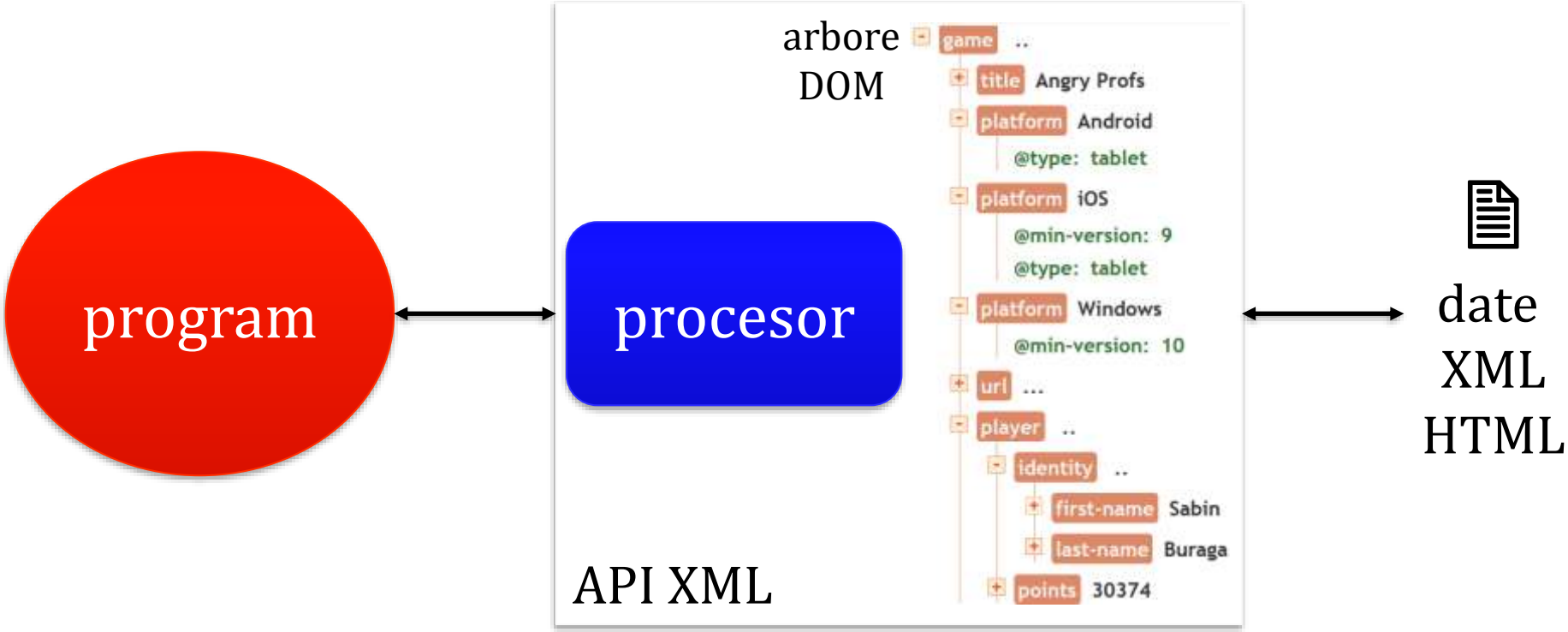
SVG DOM API – procesarea documentelor SVG în Java (Apache Batik): **xmlgraphics.apache.org/batik/**
developer.mozilla.org/Web/API/Document_Object_Model#SVG_interfaces

TwiML (*Twilio Markup Language*) API – accesarea serviciilor de telefonie Twilio via biblioteci C#, Java, Node.js, PHP, Python, Ruby: **www.twilio.com/docs/voice/twiml**

Xamarin.Forms – crearea via C# de interfețe-utilizator specificate în XAML pentru Android, iOS și Windows
dotnet.microsoft.com/apps/xamarin/xamarin-forms

dom: implementări

Procesarea documentelor XML/HTML



Studiu de caz:
prelucrarea
– recurgând la diverse limbaje de programare –
a informațiilor referitoare
la lista de proiecte propuse

deși programele diferă, maniera de procesare
este similară, datorită standardului DOM

vezi exemplele
de cod asociate
acestei prelegeri

```
$modified = 0;                          // indică dacă a fost modificat arborele DOM
$dom = new DomDocument;                 // instanțiem un obiect DOM
$dom->load("projects.xml");             // încărcăm documentul XML

$projs = $dom->getElementsByTagName("project");
foreach ($projs as $proj) {             // procesăm fiecare <project> în parte
  // preluăm nodurile element <title> și-l alegem pe primul
  $titles = $proj->getElementsByTagName("title");
  echo "<p>Proiect: <em>" . $titles[0]->nodeValue . "</em>";

  if ($proj->hasAttribute("class")) {     // este specificată clasa proiectului?
    echo " de clasa " . $proj->getAttribute("class") . "</p>";
  } else {
    echo " de clasa necunoscuta</p>";
    // nu există atributul "class", îl creăm – proiectul va fi de clasa D
    $attr = $proj->setAttribute("class", "D");
    $modified = 1;                        // marcăm arborele ca fiind modificat
  }
}

if ($modified) {  // dacă DOM-ul a fost modificat, îi afișăm reprezentarea text
  $xmldoc = $dom->saveXML();  // serializare
  echo "<pre>" . htmlentities($xmldoc) . "</pre>";
}
```

procesări DOM
în limbajul PHP

documentul XML prelucrat:

```
<?xml version="1.0" ?>
<!-- Document XML despre proiectele propuse -->
<projects>
 <project class="M">
  <title>MegaProj</title>
  <desc>...</desc>
  <stud>2</stud>
  <url>http://web.info/</url>
 </project>
 <project>
  <title>Web4all</title>
  <desc />
  <stud>3</stud>
 </project>
</projects>
```

rezultatul obținut în urma procesării DOM efectuate de programul PHP:

Proiect: *MegaProj* de clasa M
Proiect: *Web4all* de clasa necunoscuta

```
<?xml version="1.0"?>
<!-- Document XML despre proiectele propuse -->
<projects>
  <project class="M">
    <title>MegaProj</title>
    <desc>...</desc>
    <stud>2</stud>
    <url>http://web.info/</url>
  </project>
  <project class="D">
    <title>Web4all</title>
    <desc/>
    <stud>3</stud>
  </project>
</projects>
```

avansat

```
import urllib
import xml.dom

from xml.dom.minidom import parse

try:
  doc = urllib.urlopen('projects.xml')        # încărcăm documentul
  dom = parse(doc)                            # îl procesăm…
  # parcurgem elementele <project> și oferim informații despre fiecare
  for proj in dom.getElementsByTagName('project'):
    print 'Proiectul ' + proj.childNodes[1].firstChild.nodeValue + \
        ' este de clasă: ' + proj.getAttribute('class')
except Exception:
  print 'Din păcate, a survenit o excepție.'
```

procesări DOM în limbajul Python

avansat

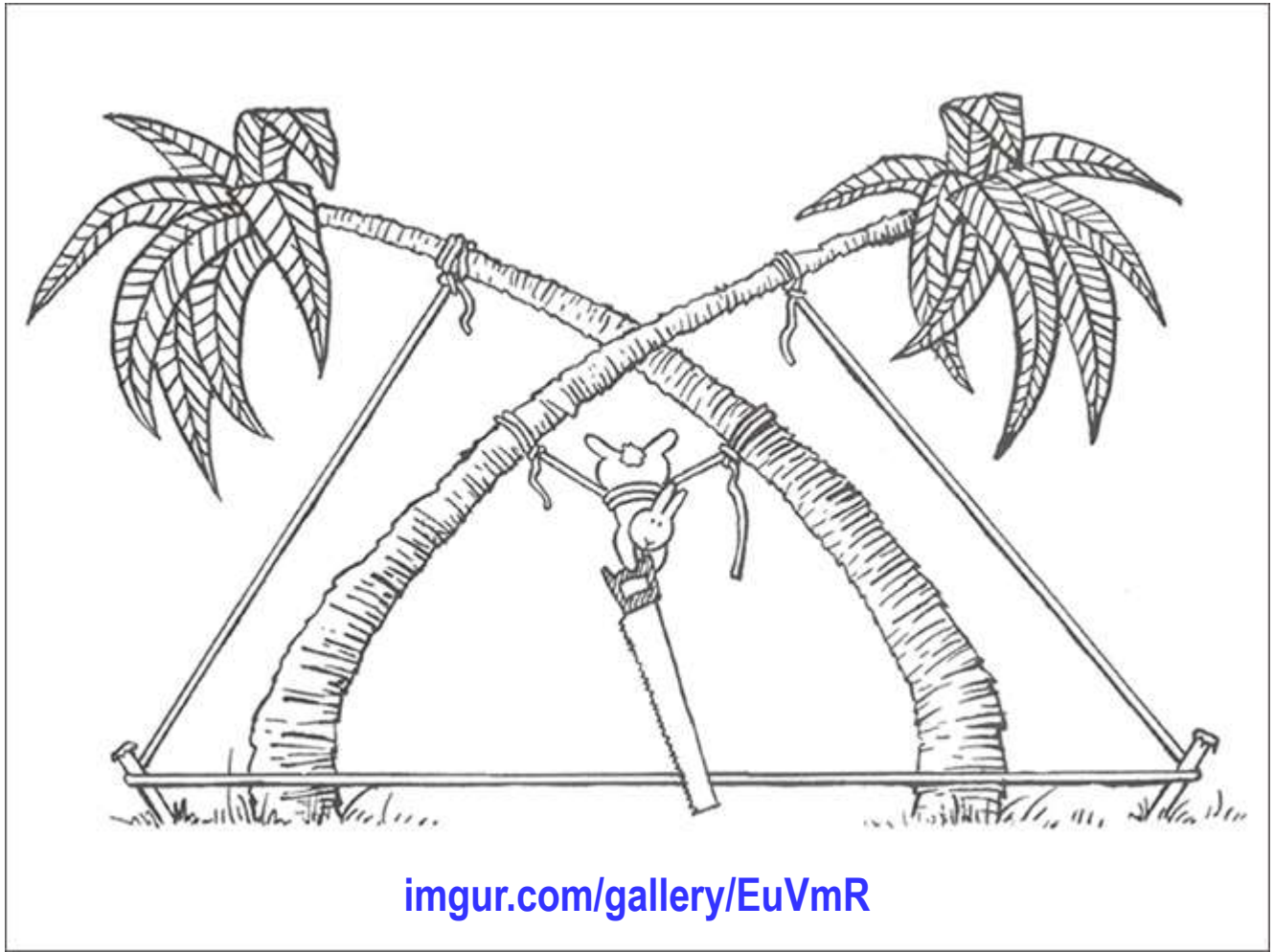
```
using System.Xml;
...
try {
  doc = new XmlDocument(); // instanțiem un document XML
  doc.Load("projects.xml");   // pentru a fi încărcat

  // afișăm informații privitoare la proiecte: titlu și clasă
  XmlNodeList projs = doc.GetElementsByTagName("project");
  foreach (XmlElement proj in projs) {
    // selectăm nodurile <title> via o expresie XPath
    XmlNodeList titles = proj.SelectNodes("./title");
    foreach (XmlElement title in titles) {
      Console.Write("Proiect: {0} ", title.InnerXml);
    }
    if (proj.HasAttribute("class") == true) { // există clasa specificată?
      Console.WriteLine("de clasa '{0}'.", proj.GetAttribute("class"));
    }
} catch ( Exception e ) {
  // a survenit o excepție…
}
```

procesări DOM
în C# (.NET)

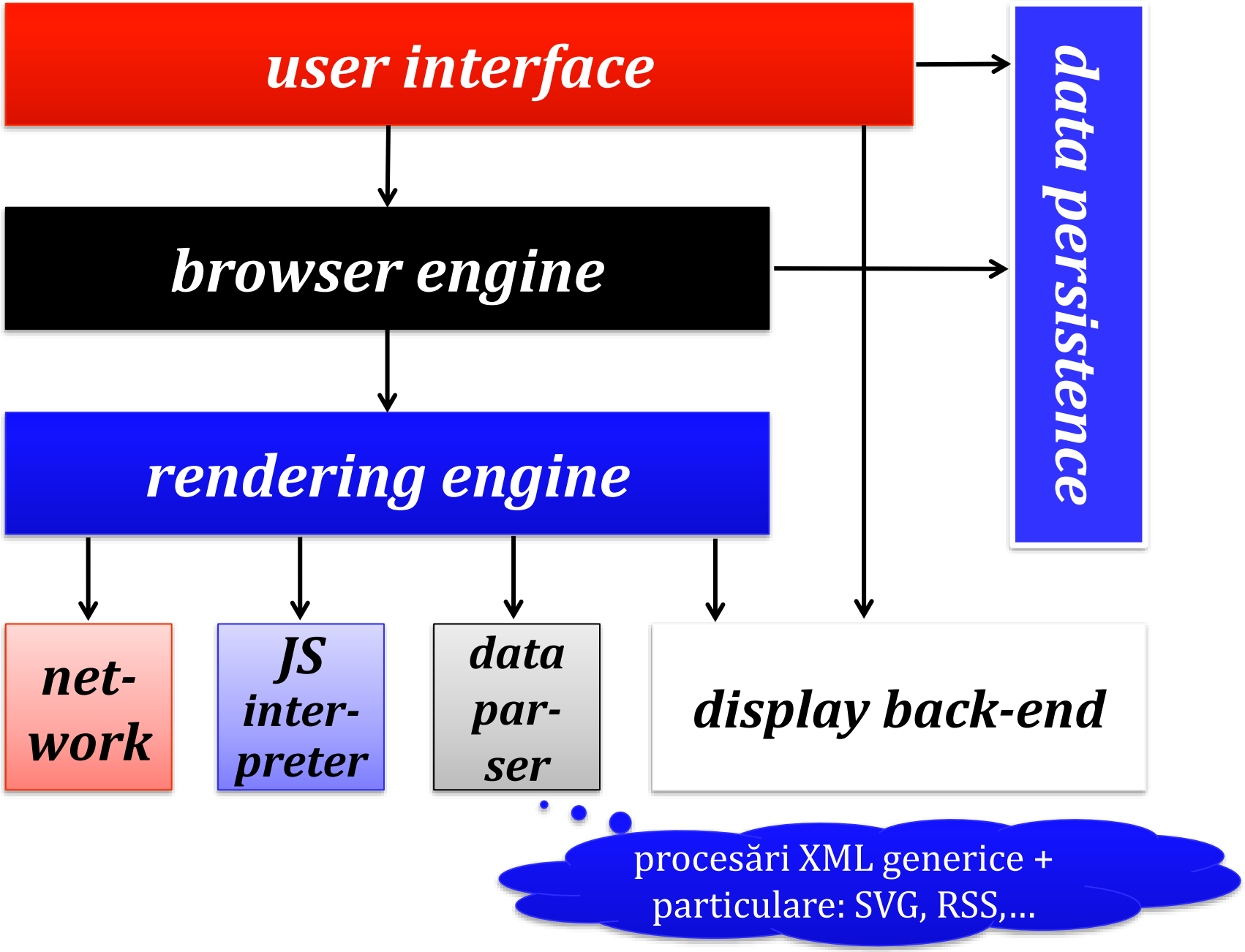
procesări DOM
în limbajul Java

```
import org.w3c.dom.*;
import javax.xml.parsers.*;
...
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
DocumentBuilder docb = dbf.newDocumentBuilder();
Document doc = docb.parse("projects.xml");
traversează (doc.getDocumentElement()); // traversăm recursiv arborele DOM
...
static private void traversează (Node nod) {
  // afișăm numele nodurilor de tip element, plus numărul de atribute
  if (nod.getNodeType() == Node.ELEMENT_NODE) {
    System.out.println ("Elementul '" + nod.getNodeName() + "' are " +
      nod.getAttributes().getLength() + " atribute.");
  }
  Node copil = nod.getFirstChild();          // parcurgem în adâncime...
  if (copil != null)
    traversează (copil);                     // există minim un copil (sub-nod)?
  copil = nod.getNextSibling();              // parcurgem în lățime...
  if (copil != null)
    traversează (copil);                     // există minim un nod sibling?
}
```

Prelucrarea prin DOM a documentelor XML/HTML la nivel de client Web?



Arhitectura unui *browser* Web



dom: browser

Accesarea/procesarea documentelor HTML și XML – fără validare – se realizează via DOM de programe JavaScript (ECMAScript) interpretate de navigatorul Web

dom: browser

Accesarea/procesarea documentelor HTML și XML – fără validare – se realizează via DOM de programe JavaScript (ECMAScript) interpretate de navigatorul Web

pentru detalii, a se studia unele dintre prelegerile materiei *Full-Stack Web Development*

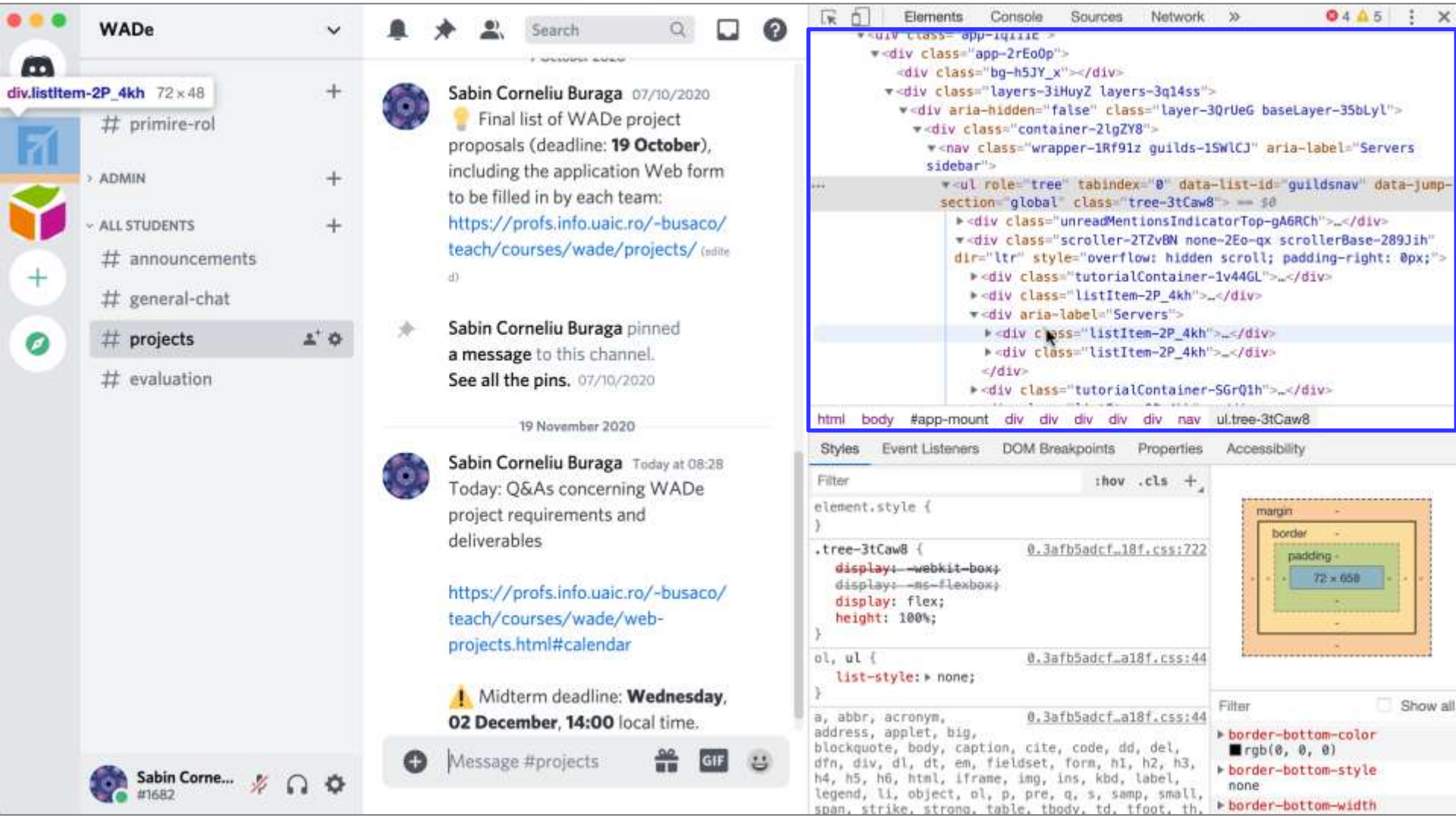
profs.info.uaic.ro/~busaco/teach/courses/staw/web-film.html

Un program JavaScript are acces la arborele DOM
(*Document Object Model*)
corespunzător documentului HTML

se recurge în principal la DOM *Living Standard*

specific HTML5, în continuă dezvoltare

Programul JS recurge la obiecte implementând interfețele DOM pentru accesarea și modificarea reprezentării interne a unui document HTML (*i.e.* arborele DOM)



inspectarea arborelui DOM asociat unui document HTML

aici, o instanță de *browser* Web în cadrul unei aplicații native

(Discord creat cu *framework*-ul Electron: Chromium + Node.js)

avansat

Shadow DOM (*shadow tree*)

arborele DOM al cărui element rădăcină e un nod (*shadow root*) al altui arbore corespunzător unui document

atașat întotdeauna unui arbore de noduri
(nu poate exista de sine-stătător,
ci doar încapsulat de un document)

avansat

shadow DOM

descris în *DOM Living Standard* (23 martie 2021)

dom.spec.whatwg.org/#shadow-trees

developer.mozilla.org/Web/Web_Components/Using_shadow_DOM

avansat

De asemenea, programele JavaScript au acces la diverse obiecte oferite de mediul de execuție pus la dispoziție de *browser*

e.g., informații privind contextul rulării (caracteristici ale navigatorului, latența rețelei), istoricul navigării, fereastra de redare a conținutului, transfer (a)sincron de date,…

The Web platform: Browser technologies

avansat

Core platform

- HTML
- DOM
- ECMAScript
- WebAssembly
- URL
- Fetch (includes CORS)
- XMLHttpRequest
- Encoding

CSS features

- CSS roadmap
- Animations
- Background-image options
- Border images
- Border radius (rounded corners)
- Box shadows
- Box sizing
- Cascading and inheritance
- Colors
- Compositing and Blending
- Containment
- Device Adaptation
- Downloadable fonts (@font-face)
- Exclusions
- Feature queries (@supports)
- Filter Effects
- Fixed positioning (position:fixed)
- Flexible box layout (Flexbox)

Graphics and typography

- Canvas
- WebGL
- SVG
- WOFF
- MathML
- Web Animations

Media

- Web Audio API
- WebVTT
- Media Source Extensions
- Media Fragments

Platform interaction, events, messaging

- Notifications API
- Cross-document messaging
- Channel messaging
- Fullscreen API
- Geolocation
- Device Orientation
- Screen Orientation
- UI Events (formerly DOM Events)
- Pointer Events
- Touch Events
- Pointer Lock
- Gamepad
- getUserMedia
- Battery Status
- Vibration

HTML bonus features

- classList (DOMTokenList)
- dataset (data-* attributes)
- async for scripts
- defer for scripts
- Session-history management
- hashchange
- Sandboxed iframe
- Drag and drop
- contentEditable
- ARIA

Performance optimization and analysis

- Web Workers
- Shared Workers
- Timing control for script-based animations
- Navigation Timing
- Page Visibility
- User Timing
- Performance Timeline
- High Resolution Time
- SIMD (Single instruction, multiple data)

Security and privacy

- Content Security Policy (CSP)
- Upgrade Insecure Requests
- Web Cryptography API
- Referrer policy
- Tracking Preference Expression (DNT)
- Web Authentication

tehnologii oferite de *browser* – **platform.html5.org**

DOM

- Credential Management 29
- Cryptography 25
- CSS 130
- Custom Elements 4
- Device Orientation 18
- Document 87
- DocumentFragment 4
- DOM 89
  - Document Object Model: Events
  - Document Object Model: Examples
  - Document Object Model: How to create ...
  - Document Object Model: Introduction
  - Document Object Model: Locating DOM ...
  - Document Object Model: Traversing an ...

DOM

When writing code for the Web with JavaScript, there are a great many APIs available. Below is a list of all the interfaces (that is, types of objects) that you may be able to use while developing your Web app or site.

Specifications

A

- Alarm API
- Ambient Light Events
- App Cache
- Application Compatibility Layer
- Apps
- Archive API
- Audio Channels API

B

- Background Tasks
- Media Source Extensions
- MediaStream Recording
- Mobile Connection API
- Mobile Messaging API
- Mozilla Payment API

N

- NFC API
- Navigation Timing
- Network Information API
- Network Stats API

detalii privind fiecare API implementat de navigatorul Web

devdocs.io/dom/

- DOMException
- DOMException.code
- DOMException.constructor

- Broadcast Channel API
- Browser API
- Performance API
- Performance Timeline API
- Permissions API

DOM

Arborele DOM asociat documentului HTML poate fi accesat/alterat via obiectul **document**

instanță a clasei implementând interfața **HTMLDocument**

```
interface HTMLDocument : Document {
  attribute DOMString title;                    // titlul documentului
  readonly attribute DOMString referrer; // adresa resursei ce referă pagina
  readonly attribute DOMString domain; // domeniul de care aparține
  readonly attribute DOMString URL;      // URL-ul absolut al documentului
  attribute HTMLElement body;             // acces la elementul <body>

  attribute DOMString cookie;             // acces la cookie-uri
                          // emite o excepție dacă e asignată o valoare

  void open ();   // deschide un flux de scriere (alterează DOM-ul curent)
  void close ();  // închide fluxul de scriere și forțează redarea conținutului
  void write (in DOMString text);     // scrie un șir de caract. (e.g., cod HTML)
  void writeln (in DOMString text);  // idem, dar inserează și New Line
  NodeList getElementsByName (in DOMString numeElement);
    // furnizează o listă de elemente conform unui nume de tag
};
```

interfață specificată în limbajul declarativ **WebIDL**

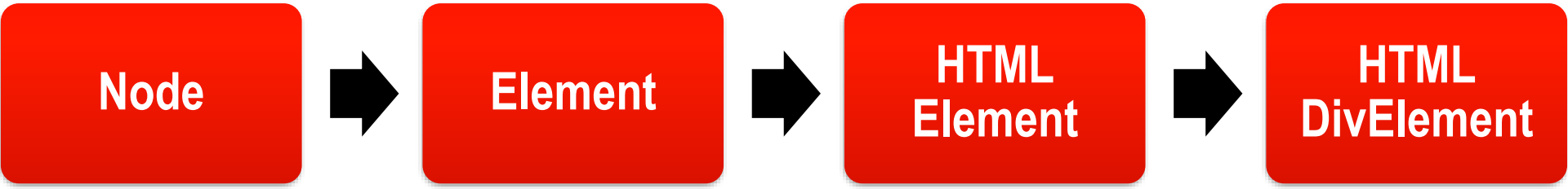
Deoarece interfața **HTMLDocument**
extinde **Document**,
putem recurge la funcționalitățile
stipulate de specificația DOM generală

Deoarece interfața **HTMLDocument**
extinde **Document**,
putem recurge la funcționalitățile
stipulate de specificația DOM generală

de exemplu, proprietatea **documentElement**
desemnează nodul-rădăcină al unui document HTML

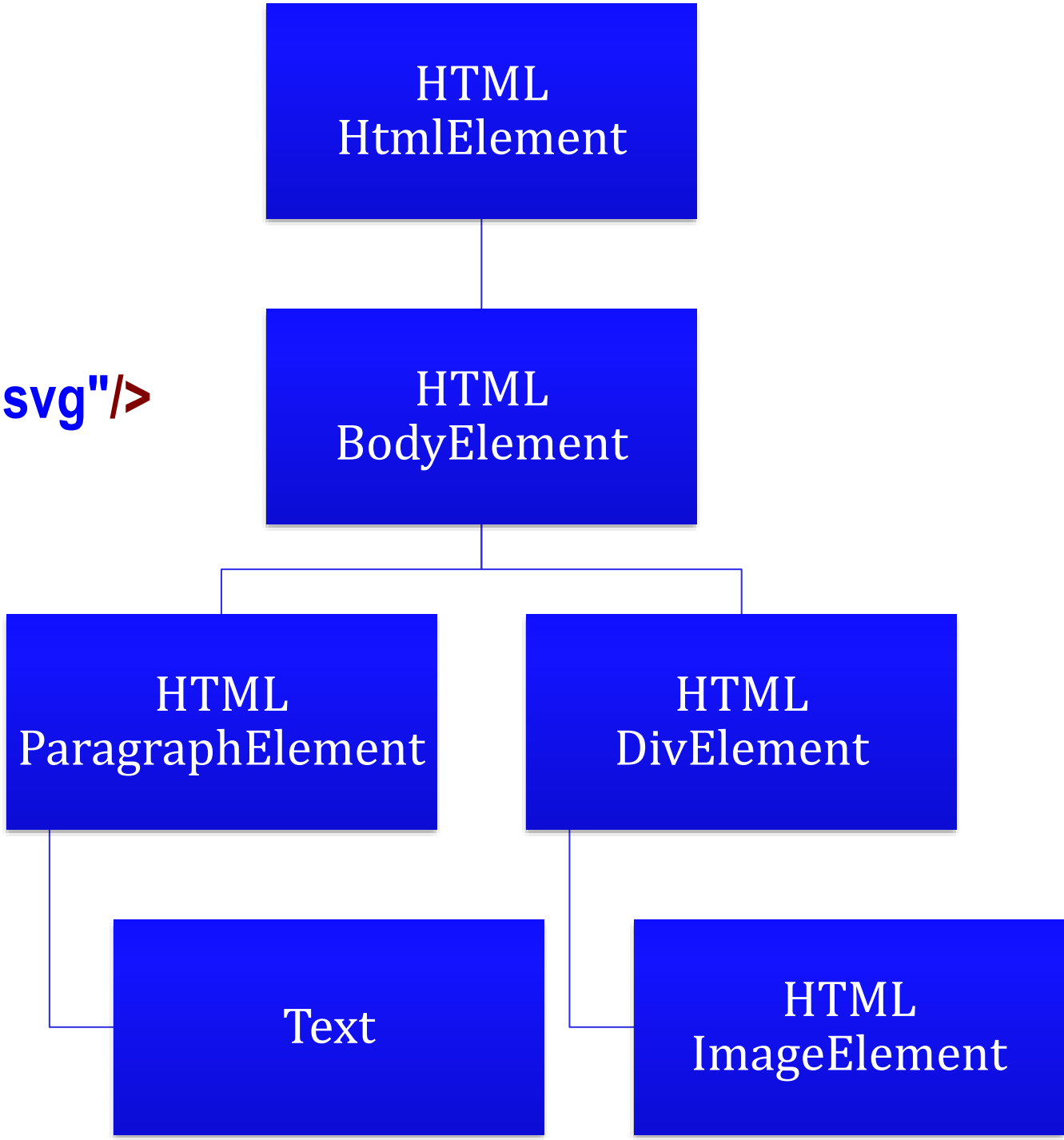
Interfața **HTMLElement** o extinde
pe cea generală oferită de DOM

fiecare element HTML specific derivă din ea

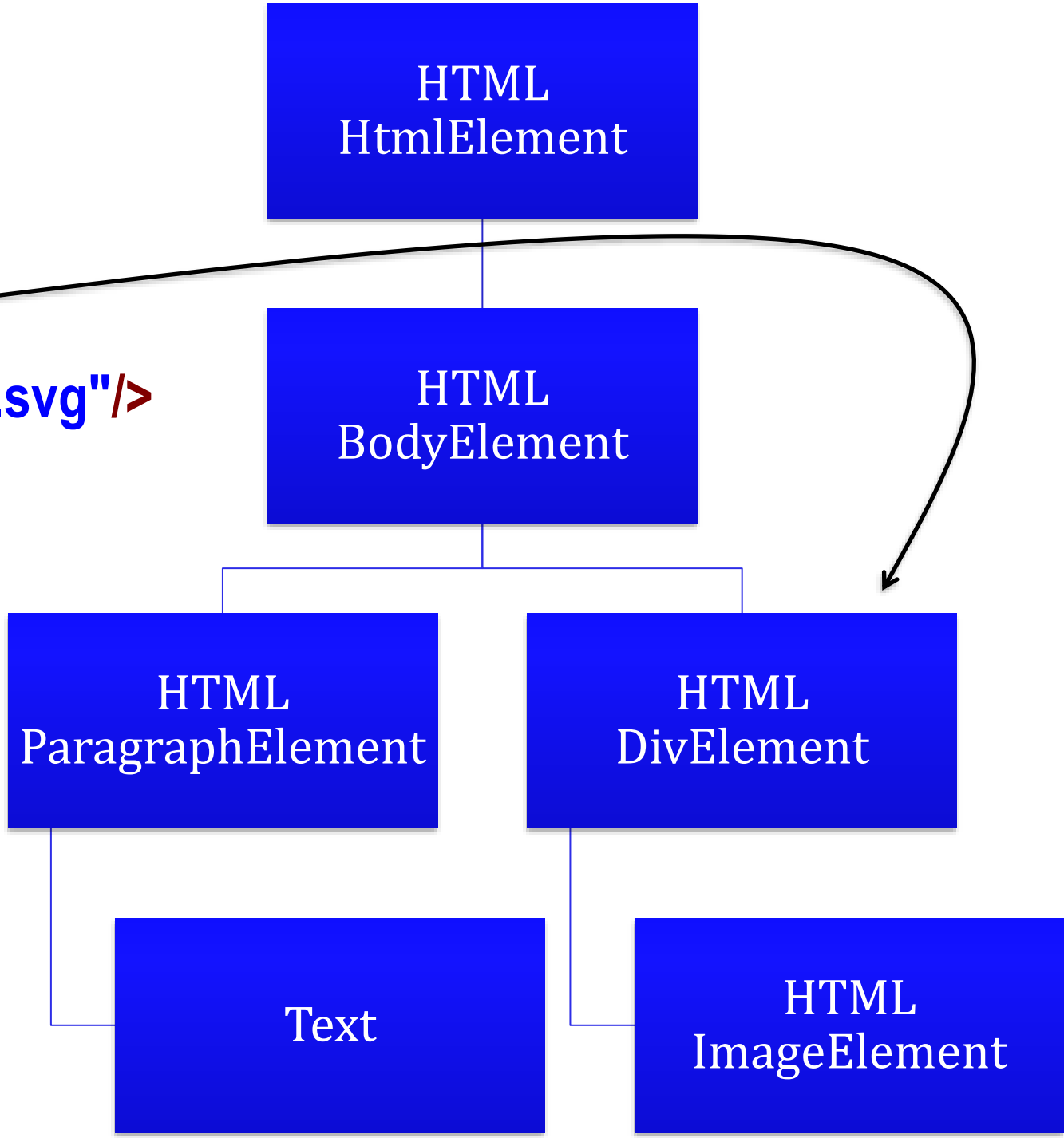


o interfață specifică fiecărui element HTML

```
<!DOCTYPE html>
<html>
  <body>
    <p>Web</p>
    <div>
      <img src="web.svg"/>
    </div>
  </body>
</html>
```



```
<!DOCTYPE html>
<html>
  <body>
    <p>Web</p>
    <div>
      <img src="web.svg"/>
    </div>
  </body>
</html>
```



```
// un element HTML generic
interface HTMLElement : Element {
   attribute DOMString      id;              // identificator asociat elementului
   attribute DOMString      title;           // titlu explicativ
   attribute DOMString      lang;            // limba în care e redactat conținutul
   attribute DOMString      className;       // numele clasei CSS folosite pentru redare
};
// specifică un formular Web
interface HTMLFormElement : HTMLElement {
   readonly attribute HTMLCollection  elements; // elementele HTML incluse în formular
   readonly attribute long  length;     // numărul câmpurilor formularului
   attribute DOMString      action;     // URI-ul resursei ce procesează datele
   attribute DOMString      enctype;    // tipul MIME de codificare a datelor
                                        // (e.g., application/x-www-form-urlencoded)
   attribute DOMString      method;     // metoda HTTP folosită: GET, POST
   void                     submit(); // trimite date URI-ului definit de 'action'
};
// o imagine (conținut grafic raster)
interface HTMLImageElement : HTMLElement {
   attribute DOMString     alt;          // text alternativ descriind conținutul grafic
   attribute DOMString     src;          // URI-ul resursei grafice
};
```

Interfața **HTMLCollection** modelează o listă de noduri

un nod poate fi accesat folosind un index numeric
sau pe baza unui identificator (*e.g.*, stabilit via atributul **id**)

```
interface HTMLCollection {
  // reprezintă lungimea listei
  readonly attribute unsigned long length;
  // oferă un nod via un index numeric
  Node      item (in unsigned long index);
  // furnizează un nod pe baza numelui
  Node      namedItem (in DOMString name);
};
```

Proprietăți specifice DOM HTML

innerHTML

proprietate – mutabilă – ce furnizează codul HTML din cadrul unui nod de tip **Element**

Proprietăți specifice DOM HTML

textContent

proprietate ce furnizează/stabilește conținutul textual al nodului și posibililor descendenți

HTML ▼

```html
<form id="unFormular">
  <!-- corpul formularului va fi generat via JavaScript -->
</form>
```

JavaScript + No-Library (pure JS) ▼

```javascript
function adaugaControl (element, tip, nume, explicatie) {
    // vom crea un element <input> atasandu-i atributele esentiale
    let control = document.createElement ('input');
    control.type = tip;
    control.id = nume;
    control.name = nume;

    // vom adauga si o eticheta explicativa
    let eticheta = document.createElement ('label');
    eticheta.htmlFor = nume;
    eticheta.innerHTML = explicatie;
    // vom altera diverse stiluri CSS
    eticheta.style.background = 'lightgray';
    eticheta.style.color = '#333';
    eticheta.style.padding = '0.2em';

    // vom plasa elementele <label> si <input> intr-un container
    let container = document.createElement ('div');
    container.className = "interactiv";
    container.appendChild (eticheta);
    container.appendChild (control);

    // vom adauga nodul container la elementul dat ca parametru
    element.appendChild (container);
}

// preluam elementul de tip formular
let formular = document.getElementById ('unFormular');
// adaugam controalele de interactiune dorite
adaugaControl (formular, 'text', 'nume', 'Numele meu: '
```

CSS ▼

JS Fiddle

```css
.interactiv {
    border-style: dotted;
    border-width: 1px;
    border-color: gray;
    width: 20em;
    padding: 1em;
    margin: 0.2em;
    font-family: sans-serif;
}
label {
    width: 50%;
    display: block;
    float: left;
    margin-right: 0.3em;
}
```

Numele meu:

Codul secret:

Particip? ☐

Web Hackathon 2018 ○

Web Hackathon 2019 ○

generare dinamică via JavaScript
a unui formular Web
jsfiddle.net/busaco/0wvn3fha/

```
function adaugaControl (element, tip, nume, explicatie) {
   // vom crea un element <input> atașându-i atributele esențiale
   let control = document.createElement ('input');
   control.type = tip;
   control.id = nume;
   control.name = nume;

   let eticheta = document.createElement ('label');    // o etichetă explicativă…
   eticheta.htmlFor = nume;
   eticheta.innerHTML = explicatie;
   eticheta.style.background = 'lightgray';                // …plus diverse stiluri CSS
   eticheta.style.color = '#333';
   eticheta.style.padding = '0.2em';

   // vom plasa elementele <label> și <input> într-un container
   let container = document.createElement ('div');
   container.className = "interactiv";
   container.appendChild (eticheta);
   container.appendChild (control);

   // vom adăuga nodul container la elementul dat ca parametru
   element.appendChild (container);
}
```

```javascript
// preluăm elementul de tip formular
let formular = document.getElementById ('unFormular');

// adăugăm controalele de interacțiune dorite
adaugaControl (formular, 'text', 'nume', 'Numele meu: ');
adaugaControl (formular, 'password', 'parola', '<em>Codul secret</em>: ');
adaugaControl (formular, 'checkbox', 'particip', 'Particip? ');
for (let an = 2018; an <= 2020; an++) {
   adaugaControl (formular, 'radio', 'an', 'Web Hackathon ' + an);
}
```

```html
<form id="unFormular">
  <!--corpul formularului va fi generat via JavaScript-->
▼ <div class="interactiv">
    <label for="nume" style="background: lightgray none repeat scroll 0% 0%; color: rgb(51, 51, 51);
    padding: 0.2em;">Numele meu:</label>
    <input id="nume" type="text" name="nume">
  </div>
▶ <div class="interactiv">···</div>
▶ <div class="interactiv">···</div>
▶ <div class="interactiv">···</div>
▶ <div class="interactiv">···</div>
▶ <div class="interactiv">···</div>
</form>
```

arborele DOM corespunzător codului HTML generat prin program

Suport pentru procesarea proprietăților CSS

pe baza unui model obiectual specific
CSSOM (*CSS Object Model*)

specificație în lucru (*draft*) – 18 ianuarie 2021

drafts.csswg.org/cssom/

Suport pentru procesarea proprietăților CSS

actualmente, modificarea proprietăților de stil se poate realiza via proprietatea **HTMLElement.style**

```
// asocierea mai multor stiluri CSS
elem.style.cssText = "color: blue; border: 1px solid #000";
// similar cu linia – are drept efect alterarea arborelui DOM:
elem.setAttribute ("style", "color: blue; border: 1px solid #000;");
```

developer.mozilla.org/Web/API/CSSStyleDeclaration

Găsirea de noduri via selectori CSS

Selectors API

inițial, recomandare W3C (2013)

actualmente, parte din *DOM Living Standard* (2021)

Găsirea de noduri via selectori CSS

querySelector (*selectori*)
furnizează primul element – folosind traversarea în adâncime în pre-ordine – care se potrivește grupului de selectori (delimitați de virgulă)

querySelectorAll (*selectori*)
oferă lista de tip NodeList a tuturor elementelor găsite

Găsirea de noduri via selectori CSS

```
// toate elementele <li> selectate via CSS (date de tip NodeList)
let elemente = document.querySelectorAll ("ul.menu > li");
for (var i = 0; i < elemente.length; i++) {
  prelucrează (elemente.item (i));      // procesăm fiecare nod
}
```

★ săptămâna 6

- Curs: **Modelarea datelor. Familia XML. Validări cu DTD**

  data, modeling, semi-structure, XML, xmlns, validation, DTD, examples

- Demo: **Exemple de documente XML** (arhivă `zip`, 23K), **Exemple de validări folosind DTD** (arhivă `zip`, 4K) + studii de caz: **RSS**, **SVG** (MDN) + **Can I Use SVG?**, **Sitemaps XML Format**, **DocBook**, **KML** (Google), **XML Periodic Table**

Inspector | Console | Debugger | Network | Style Editor

Run { }

```
1 document.querySelectorAll
  ("section[id^=\"week\"]:nth-
  child(even) > h2");
2
```

selectori CSS3

Filter Output

Errors | Warnings | Logs | Info | Debug | CSS | XHR | Requests

```
» document.querySelectorAll ("section[id^=\"week
  \"]:nth-child(even) > h2");
← ▼ NodeList(3)
    ▶ 0: <h2 class="film">
    ▶ 1: <h2 class="film">
    ▶ 2: <h2 class="film">
      length: 3
    ▶ <prototype>: NodeListPrototype { item: item(),
  keys: keys(), values: values(), … }
```

exemplificare – folosim consola *browser*-ului Web:

document.querySelectorAll ("section[id^=\"week\"]:nth-child(even) > h2");

Tratarea evenimentelor

definirea de activități (*callback*-uri) executate
la apariția unui eveniment

eveniment = acțiune produsă în cadrul mediului de execuție în urma căreia programul va putea reacționa

Tratarea evenimentelor

codul JavaScript invocat la apariția unui eveniment
va putea fi încapsulat într-o funcție de tratare a acestuia
(*event handler*)

Tratarea evenimentelor

tradițional, se atașează cod JavaScript ce va fi executat la apariția unui eveniment de bază (*e.g.*, **onclick**, **onmouseover**, **onchange**, **onload**, **onkeypress**,...) asupra unui element

Tratarea evenimentelor

```
<button onclick="alert ('Au!');">Apasă-mă!</button>
```

Tratarea evenimentelor

```
<button onclick="alert ('Au!');">Apasă-mă!</button>
```

pentru a inhiba execuția acțiunii implicite, codul JavaScript va trebui să întoarcă *false*

Tratarea evenimentelor

mai „evoluat” via DOM 1:

```
document.getElementById ("identificator").onclick
= trateazaClick;
```

Tratarea evenimentelor

tratarea standardizată a evenimentelor:
minimal, conform specificației DOM Level 2 Events

www.w3.org/TR/DOM-Level-2-Events/

Tratarea evenimentelor

specificarea de activități executate
la apariția unui eveniment

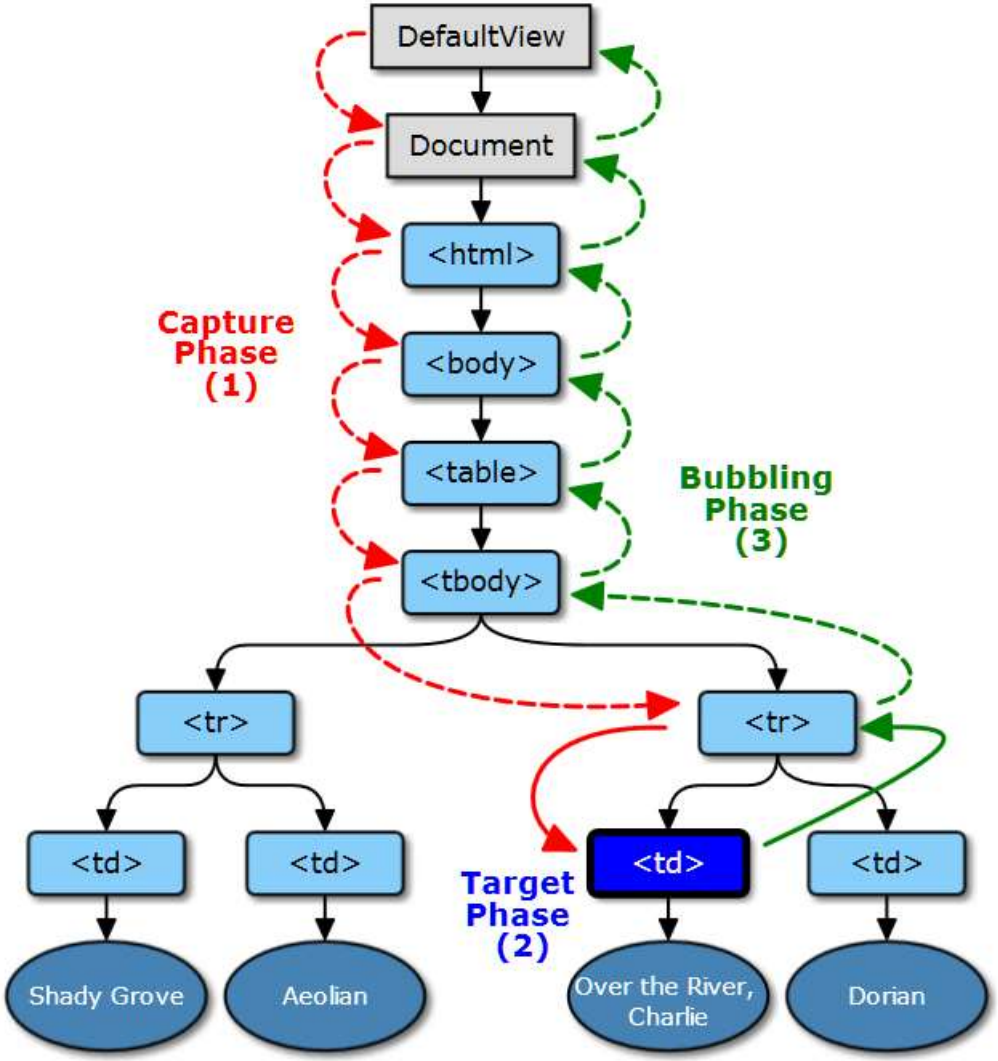
obiect.addEventListener ("*eveniment*", *funcție*, *mod*);

Tratarea evenimentelor

descrierea arborescentă a fluxului de evenimente

capture versus bubble

de parcurs **javascript.info/bubbling-and-capturing**



fluxul de evenimente (T. Leithead *et al.*, 2012)

a se studia și W. Page, *An Introduction to DOM Events* (2013)
www.smashingmagazine.com/2013/11/an-introduction-to-dom-events/

Tratarea evenimentelor

descrierea arborescentă a fluxului de evenimente

mod = true

se încearcă tratarea evenimentului pornind
de la rădăcină până la obiectul-țintă – *capture phase*

Tratarea evenimentelor

descrierea arborescentă a fluxului de evenimente

mod = false

se încearcă tratarea evenimentului atunci când evenimentul e propagat de la obiectul unde a survenit până la entitățile superioare lui – *bubbling phase*

Tratarea evenimentelor

se va utiliza un set standard de evenimente

Tipuri de evenimente – interacțiune cu utilizatorul

mouse: **click dblclick mouseenter mousedown mouseup mouseover mousemove contextmenu select wheel**

keyboard: **keypress keydown keyup**

clipboard: **copy cut paste**

view: **resize scroll fullscreenchange fullscreenerror**

drag & drop: **dragstart drag dragenter dragover dragleave dragend drop**

form: **focus blur select submit reset**

document (page lifecycle): **load DOMContentLoaded beforeunload unload abort cancel**

avansat

Tipuri de evenimente – vizând resursele

network: **online offline**

session history: **pagehide pageshow popstate**

printing: **beforeprint afterprint**

media: **canplay play playing pause suspend waiting seeking seeked stalled complete ended emptied durationchange ratechange volumechange timeupdate** ...

progress: **loadstart progress error timeout abort load loaded**

storage: **change storage**

Web socket: **open message error close**

avansat

Tipuri de evenimente – altele

CSS transitions: **transitionstart transitionrun transitionend transitioncancel**

CSS animations: **animationstart animationend animationiteration**

value change: **broadcast hashchange input readystatechange**...

pentru amănunte, a se explora și
developer.mozilla.org/Web/Events

avansat

Tipuri de evenimente – specifice HTML5

asociate API-urilor disponibile:

Ambient Light, *Bluetooth*, *Canvas*, *Clipboard*, *Device Orientation*, *File System*, *IndexedDB*, *Media Capture & Streams*, *Payment*, *Pointer*, *Proximity*, *Push*, *SVG*, *Touch*, *Time & Clock*, *TV*, *Web Animations*, *Web Audio*, *WebGL*, *WebRTC*, *Web Notifications*, *Web Workers*, *Websockets*, *Web Storage*, *WebXR Device* etc.

developer.mozilla.org/docs/Web/API

Tratarea evenimentelor

proprietăți utile ale obiectului **Event**

type

specifică tipul evenimentului ca șir de caractere
e.g., **"click"**, **"load"**, **"scroll"**, **"submit"**

Tratarea evenimentelor

proprietăți utile ale obiectului **Event**

target

desemnează nodul
asupra căruia evenimentul a fost declanșat inițial

Tratarea evenimentelor

proprietăți utile ale obiectului **Event**

currentTarget

indică nodul care tratează evenimentul

Tratarea evenimentelor

proprietăți utile ale obiectului **Event**

bubbles

indică dacă evenimentul se propagă
spre elemente ascendente (valoarea *true*)
ori către descendenți (valoarea *false*)

Tratarea evenimentelor

proprietăți utile ale obiectului **Event**

bubbles

e.g., evenimentele abort, error, select, submit, resize, scroll, click, mousedown, mouseover, mousemove, mouseout, touchstart, touchend pot avea **bubbles** = *true*

Tratarea evenimentelor

proprietăți utile ale obiectului **Event**

bubbles

în cazul evenimentelor focus, blur, load, unload, proprietatea **bubbles** are valoarea *false*

Tratarea evenimentelor

proprietăți utile ale obiectului **Event**

cancelable

precizează dacă evenimentul poate fi întrerupt

Tratarea evenimentelor

proprietăți utile ale obiectului **Event**

cancelable

de exemplu, pentru evenimentele load, unload, abort, error, select, focus, blur, resize, scroll, touchcancel
proprietatea **cancelable** este setată ca fiind *false*

Tratarea evenimentelor

proprietăți utile ale obiectului **Event**

cancelable

pentru evenimente precum click, mousedown, mouseup, mouseover, mousemove, mouseout, touchstart, touchend, touchmove proprietatea **cancelable** poate fi *true*

Tratarea evenimentelor

eliminarea tratării unui eveniment

removeEventListener ()

Tratarea evenimentelor

ignorarea comportamentului implicit

preventDefault ()

Tratarea evenimentelor

trimiterea evenimentului să poată fi procesat conform modelului oferit de implementare

dispatchEvent ()

Tratarea evenimentelor

oprirea propagării unui eveniment
în cadrul arborelui DOM

stopPropagation ()

avansat

```
<html>
<head><title>Evenimente JS</title></head>
<body>
  <form>
    <textarea id="editor"></textarea>
    <input type="text" id="text" />
  </form>
</body>
<script type="application/javascript">
const infoEv = e => { console.log (
  `Eveniment: ${e.type}\n
  Caracter: ${String.fromCharCode (e.charCode)}\n
  Element țintă al evenim.: ${e.target}\n
  Ținta curentă: ${e.currentTarget}`);
};
document.addEventListener("keypress", infoEv);
</script>
</html>
```

adaptare după Baptiste Pesquet (2017)

github.com/bpesquet/thejsway/blob/master/manuscript/chapter16.md

exemplificare: afișarea datelor vizând un eveniment

avansat

W

e

b

Inspector | Console | Debugger | Network | DOM

Filter Output | Errors | Warnings | Logs | Info | Debug | CSS | XHR | Requests

```
Eveniment: keypress                                          keypress.html:17:31

  Caracter: W

  Element țintă al evenim.: [object HTMLTextAreaElement]

  Ținta curentă: [object HTMLDocument]
Eveniment: keypress                                          keypress.html:17:31

  Caracter: e

  Element țintă al evenim.: [object HTMLInputElement]

  Ținta curentă: [object HTMLDocument]
Eveniment: keypress                                          keypress.html:17:31

  Caracter: b

  Element țintă al evenim.: [object HTMLBodyElement]

  Ținta curentă: [object HTMLDocument]
```

apăsarea unei taste în câmpul <textarea>

apăsarea unei taste în câmpul <input>

apăsarea unei taste în corpul paginii Web

avansat

```
// Adaptare după https://eloquentjavascript.net/14_event.html
const trateazaEveniment = ev => {
   // plasăm un 'punct' la coordonatele cursorului mouse-ului
   let pct = document.createElement ('div');
   pct.className  = (ev.type === 'dblclick') ? 'punct roz' : 'punct';
   pct.style.left = (ev.pageX - 5) + 'px';
   pct.style.top  = (ev.pageY - 5) + 'px';
   document.body.appendChild (pct);
   console.log (`${ev.type}: Am plasat un punct la coord.
                                   (${ev.pageX}, ${ev.pageY}).`);
};
// "ascultăm" evenimentele click și dblclick
document.addEventListener ('click', trateazaEveniment);
document.addEventListener ('dblclick', trateazaEveniment);
```

vezi exemplul complet în arhivă

exemplu: tratarea evenimentelor **click** și **dblclick**

Un click (dublu) de mouse, te rog...

div.punct | 10×10

Elements Console Sources Network Performance Memory Application

```
<!DOCTYPE html>
<html>
▶<head>…</head>
…▼<body> == $0
  <p>Un click (dublu) de mouse, te rog...</p>
  ▶<script type="application/javascript">…
  </script>
  <div class="punct" style="left: 311px; top: 107px;"></div>
  <div class="punct" style="left: 143px; top: 181px;"></div>
  <div class="punct" style="left: 143px; top: 181px;"></div>
  <div class="punct roz" style="left: 143px; top: 181px;"></div>
  <div class="punct" style="left: 95px; top: 71px;"></div>
  <div class="punct" style="left: 694px; top: 22px;"></div>
</body>
```

html body

Styles Computed Event Listeners

Ancestors All Framework listeners

```
▼click
  ▼document                                  clicks.html:25
    useCapture: false
    passive: false
    once: false
    ▼handler: ev => {…}
      ▶[[Scopes]]: Scopes[2]
      [[FunctionLocation]]: <unknown>
      ▶__proto__: ƒ ()
      name: "trateazaEveniment"
      length: 1
      caller: (...)
      arguments: (...)
▶dblclick
```

dom: browser

Se oferă suport și pentru transfer asincron de date între client (*browser*) și server Web

AJAX – *Asynchronous JavaScript And XML*
via obiectul **XMLHttpRequest** și/sau **Fetch API**

vezi cursurile viitoare

avansat

Remarcă:

unele navigatoare acceptă tratarea unor evenimente nestandardizate (încă) de Consorțiul Web

avansat

Remarcă:

pot fi specificate și evenimente definite de programator

CustomEvent

dom.spec.whatwg.org/#interface-customevent
developer.mozilla.org/Web/API/CustomEvent

avansat

```
<html class="pc firefox74 js" lang="en"> event
  <!--<![endif]-->
  ▶ <head> ··· </head>
  ▼ <body class="room-editor editor stat
    event flex
    ▶ <header id="main-header" class="ma
    ▶ <div id="item-settings-modal" clas
      item-settings-modal-half"> ··· </div
    ▶ <div class="tour-modals"> ··· </div>
    ▶ <div id="tour-overlay" class="tour
    ▶ <div class="page-wrap twilight"> ··
      <div id="asset-bin-goes-here"></di
    ▶ <footer id="react-pen-footer" clas
    ▶ <div id="keycommands" class="modal
      <div id="popup-overlay" class="ove
      <div id="modal-overlay" class="ove
      <div id="react-popups" class="reac
    ▶ <noscript> ··· </noscript>
      <input id="init-data" type="hidder
      {"device":"unknown","mobile":null,
      44142f30738f4cd07a197048c9b50dba77
      <script src="https://static.codepe
      1963…406ae47d3176af996336c5d219acc
```

beforeunload	…1c296bb83b19045bb800fcac2c145a26.js:2:13007	Bubbling	DOM2
blur	…cks/js/vendor-e6b2172b3357b586708c.chunk.js:33:28433	Bubbling	DOM2
blur	…ks/js/vendor-e6b2172b3357b586708c.chunk.js:33:28433	Capturing	DOM2
change	…/js/vendor-e6b2172b3357b586708c.chunk.js:33:28433	Bubbling	DOM2
click	…ks/js/vendor-e6b2172b3357b586708c.chunk.js:33:28433	Bubbling	DOM2
click	…ks/js/vendor-e6b2172b3357b586708c.chunk.js:33:28433	Bubbling	DOM2
CONSOLE_CLOSED	…e6b2172b3357b586708c.chunk.js:33:28433	Bubbling	DOM2
CONSOLE_OPENED	…e6b2172b3357b586708c.chunk.js:33:28433	Bubbling	DOM2
CONSOLE_OPENED	…e6b2172b3357b586708c.chunk.js:33:28433	Bubbling	DOM2
CONSOLE_SERVER_CHANGE	…3357b586708c.chunk.js:33:28433	Bubbling	DOM2
DOMContentLoaded	…b2172b3357b586708c.chunk.js:33:28433	Capturing	DOM2
error	…ks/js/vendor-e6b2172b3357b586708c.chunk.js:33:31509	Bubbling	DOM2
focus	…s/js/vendor-e6b2172b3357b586708c.chunk.js:33:28433	Capturing	DOM2

inspectarea – via instrumentele incluse în *browser* – a evenimentelor standard + definite de programator (cele scrise doar cu majuscule), plus a modului de propagare: *bubbling versus capturing*

exemplu concret: **codepen.io**

avansat

dom: resurse suplimentare

Marijn Haverbeke, *Eloquent JavaScript* (*3rd Edition*), 2018
– capitolul "*The Document Object Model*"
eloquentjavascript.net/14_dom.html

The Modern JavaScript Tutorial, 2021
– secțiunea "*DOM Tree*"
javascript.info/dom-nodes

EventTarget
developer.mozilla.org/Web/API/EventTarget

avansat

dom: web storage

Web Storage

oferă mecanisme de stocare (persistentă) a datelor la nivel de client (*browser*) Web

specificație *HTML Living Standard* (24 martie 2021)
html.spec.whatwg.org/multipage/webstorage.html

avansat

dom: web storage

Interfața **Storage** oferă acces la o listă de perechi (*items*) de forma cheie—valoare

avansat

```
interface Storage {
   // mărimea spațiului de stocare
   readonly attribute unsigned long length;
   // cheie pentru acces la date
   DOMString? key(unsigned long index);
   // consultarea unui item
   getter DOMString? getItem(DOMString key);
   // asignarea de valori pentru un item
   setter undefined setItem(DOMString key, DOMString value);
   // eliminarea unui item
   deleter undefined removeItem(DOMString key);
   // ștergerea spațiului de stocare
   undefined clear();
};
```

metodele **setItem ()** și **removeItem ()**
se consideră ca fiind atomice

avansat

dom: web storage

Modul de stocare a *item*-ilor se precizează via atributele **sessionStorage** și **localStorage**

avansat

dom: web storage

Modul de stocare a *item*-ilor se precizează via atributele **sessionStorage** și **localStorage**

stocare nepersistentă (suport pentru sesiuni)

avansat

dom: web storage

Modul de stocare a *item*-ilor se precizează via atributele **sessionStorage** și **localStorage**

alternativă la *cookie*-uri

avansat

dom: web storage

Orice modificare a zonei de stocare conduce la emiterea evenimentului **storage**

```
interface StorageEvent : Event {
    readonly attribute DOMString? key;
    readonly attribute DOMString? oldValue;
    readonly attribute DOMString? newValue;
    readonly attribute DOMString   url;
    readonly attribute Storage?     storageArea;
};
```

avansat

```
<textarea id="editor" placeholder="Începeți să tastați..."></textarea>

<script>
let editor = document.querySelector ('#editor');
// când primește controlul interacțiunii (evenimentul focus),
// <textarea> va include conținutul din localStorage – item-ul valoare
editor.addEventListener ('focus', (eveniment) => {
  if (localStorage.getItem('valoare'))
    editor.textContent = localStorage.getItem('valoare');
}, false);
// când se părăsește elementul <textarea> (evenimentul blur),
// tot ce a fost introdus este salvat în localStorage – item-ul valoare
editor.addEventListener ('blur', (eveniment) => {
  localStorage.setItem('valoare', editor.value);
}, false);
</script>
```

avansat

Web 2...

textarea#editor 177.667 × 50.6667

Inspector | Console | Debugger | Network | Style Editor | Storage | DOM

Indexed DB

Local Storage

file:///N:/salvare-text.html

Session Storage

Filter Items

Key	Value
valoare	Web 2...

Filter Output

```
focus { target: textarea#editor , isTrusted: true, view: Window, detail: 0, layerX: 0, layerY:
blur { target: textarea#editor , isTrusted: true, view: Window, detail: 0, layerX: 0, layerY:
focus { target: textarea#editor , isTrusted: true, view: Window, detail: 0, layerX: 0, layerY:
blur { target: textarea#editor Click to select the node in the inspector ndow, detail: 0, layerX: 0, layerY:
```

inspectarea în *browser* a zonei *Local Storage*
alterată de funcțiile de tratare a evenimentelor **focus** și **blur**
axate asupra controlului **<textarea>**

avansat

dom: web storage

Față de *cookie*-uri, datele stocate în **localStorage**
nu au un timp de viață stabilit *a-priori*
(pot fi șterse explicit de utilizator sau prin program)

nu este precizată o dimensiune maximă
pentru stocarea locală a datelor cu **localStorage**
(navigatorul Web poate impune o anumită valoare,
eventual ulterior modificată de utilizator)

avansat

dom: web storage

Datele memorate în **localStorage**
sunt disponibile numai la nivel local – în *browser*

serverul Web nu le poate accesa direct decât
via un program ce le transferă explicit prin HTTP(S)

avansat

dom: atribute proprii

Atribute definite de programator
(*custom HTML5 attributes*)

într-un document HTML pot fi incluse atribute proprii prefixate cu **data-** și manipulate via **HTMLElement.dataset**

pot stoca valori private
disponibile la nivel de pagină/aplicație Web

developer.mozilla.org/Learn/HTML/Howto/Use_data_attributes

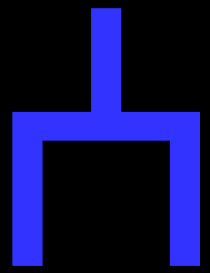
```
<div id="person" data-id="69" data-user="tux" data-date-of-birth>
Tuxy Pinguinnesscool</div>

<script>
let elem = document.querySelector ('#person'); // obținem datele
// elem.id == 'person'
// elem.dataset.id === '69'
// elem.dataset.dateOfBirth === '' (data de naștere n-a fost precizată)
elem.dataset.dateOfBirth = '2001-04-01'; // stabilim când s-a născut
// includem un alt atribut propriu stabilit prin program
elem.dataset.size = 'big';
</script>
```

```
>> elem
<- ▸ <div id="person" data-id="69" data-user="tux" data-date-of-birth="2001-04-01" data-size="big">
>> elem.dataset.dateOfBirth
<- "2001-04-01"
```

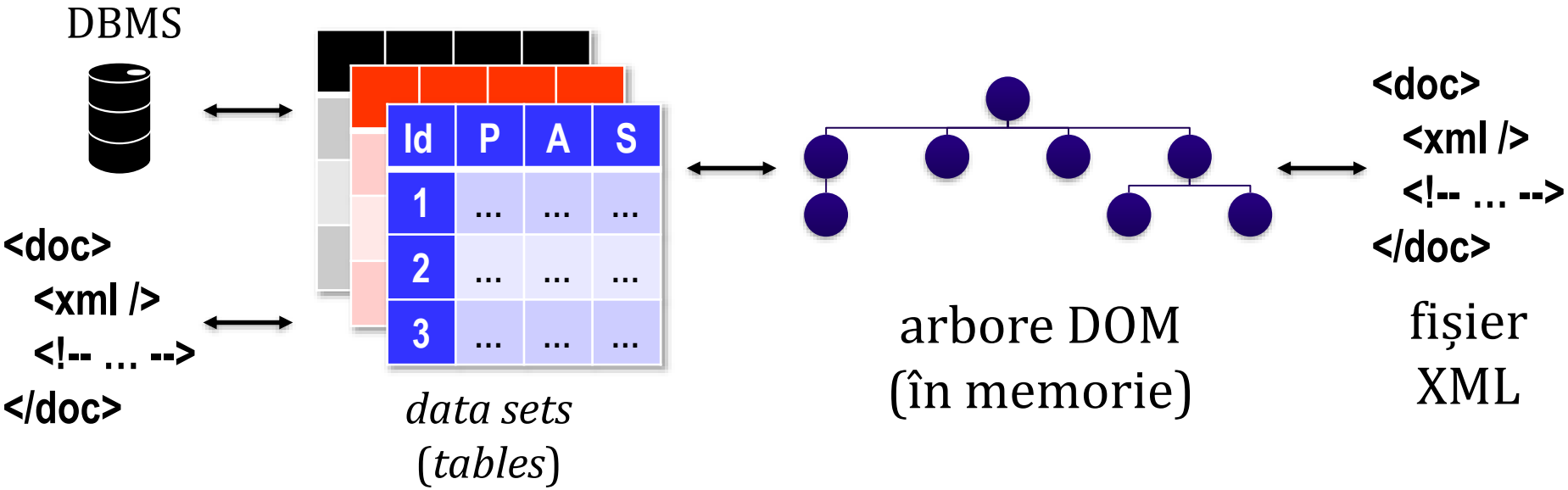
folosirea consolei navigatorului Web pentru accesarea și modificarea atributelor **data-** pe baza **dataset**

rezumat

Acces la XML/HTML prin **DOM**



caracterizare, specificare, interfețe, XPath, evenimente, *Web Storage*, data-, exemple



episodul viitor:

procesări XML via SAX + prelucrări simplificate

HTML scraping